



**UNIVERSIDAD
DE BURGOS**

**Fuentes de Datos Biomédicos y Web Semántica
(#8235)**

**Apuntes de Prácticas
Grado en Ingeniería de la Salud**

Dr. Antonio Canepa-Oneto

Curso 2026 / 2027

Índice general

Prefacio	1
Estructura del libro	1
Cómo usar este libro	2
Licencia	2
1 Práctica GitHub	3
1.1 Objetivos	3
1.2 ¿Por qué control de versiones en ciencia de datos?	3
1.3 Preparación del directorio de trabajo	4
1.4 Comandos básicos de Git	6
1.4.1 Inicializar un repositorio	6
1.5 Modificación de ficheros	8
1.5.1 Modificar archivos existentes	8
1.5.2 Borrar archivos y recuperarlos	11
1.6 Resumen de comandos	18
1.7 Licencia	19
2 Seminario, GitHub y RMarkdown/Quarto	28
2.1 Objetivos	28
2.2 Documentos dinámicos: RMarkdown y Quarto	28
2.2.1 Anatomía de un documento	29
2.2.2 Ejemplo mínimo	30
2.3 GitHub como repositorio remoto	30
2.4 Estructura del seminario evaluable	31
2.4.1 Introducción	31
2.4.2 Objetivo general y objetivos específicos	31
2.4.3 Metodología y resultados	32
2.4.4 Conclusiones generales	32
2.4.5 Referencias	32
2.5 Criterios de evaluación	32
3 Objetos en R	33
3.1 Objetivos	33
3.2 El sistema de objetos de R	33
3.3 Vectores	33
3.3.1 Vectores atómicos	33
3.3.2 Valores ausentes (NA)	34
3.3.3 Coerción (<i>coercion</i>)	35
3.3.4 Nombres	35
3.3.5 Dimensiones	36
3.3.6 Indexación de vectores	36
3.3.7 Factores	37
3.3.8 Fechas	37
3.3.9 Fechas-horas	38

3.3.10	Duraciones	38
3.4	Matrices y arrays	38
3.4.1	Coerción en matrices	38
3.4.2	Indexación de matrices	39
3.5	Tablas de datos (<code>data.frame</code>)	39
3.5.1	<code>tibble</code> frente a <code>data.frame</code>	39
3.5.2	Indexación de <code>data.frame</code>	40
3.6	Listas	40
3.6.1	Combinación de listas	41
3.6.2	Coerción de listas	41
3.6.3	Indexación de listas	41
4	Funciones en R	42
4.1	Objetivos	42
4.2	Estructuras condicionales	42
4.2.1	<code>if</code>	42
4.2.2	<code>if-else</code>	42
4.2.3	<code>ifelse()</code> : la versión vectorizada	43
4.3	Iteración	43
4.3.1	La familia <code>apply</code>	43
4.3.2	<code>while</code> : iteración condicionada	44
4.3.3	<code>for</code> : iteración sobre una secuencia conocida	45
4.4	Funciones propias	46
4.4.1	Anatomía de una función en R	46
4.4.2	Ejemplo: análisis de secuencias genéticas	46
4.4.3	Modularizar código: <code>source()</code>	48
5	Importación y exportación de datos	49
5.1	Objetivos	49
5.2	Datos estructurados: hojas de cálculo y CSV	49
5.2.1	Importar desde Excel	49
5.2.2	Importar desde CSV	49
5.2.3	Importar datos directamente desde la web	50
5.3	Datos semiestructurados: por qué importan en biomedicina	51
5.4	JSON: JavaScript Object Notation	51
5.4.1	Sintaxis y modelo de datos	51
5.4.2	JSON frente a las estructuras de datos de R	52
5.4.3	El paquete <code>jsonlite</code>	53
5.4.4	JSON anidado: el paquete <code>tidyjson</code>	55
5.4.5	Buenas prácticas al trabajar con JSON	57
5.5	XML	57
5.6	Exportación de datos	58
5.6.1	Guardar objetos de R: formato nativo (<code>.rds</code>)	58
5.6.2	Exportar a texto plano (CSV)	59
5.6.3	Exportar a JSON	59
5.6.4	Exportar a XML	59
6	Procesamiento y manejo de datos	60
6.1	Objetivos	60
6.2	Verbos de una tabla: la dimensión de las filas	60
6.2.1	<code>filter()</code> : seleccionar filas por condición	60
6.2.2	<code>slice()</code> y su familia: seleccionar filas por posición	60

6.2.3	<code>arrange()</code> : ordenar filas	61
6.2.4	<code>summarise()</code> : reducir una tabla a un resumen	61
6.3	Verbos de una tabla: la dimensión de las columnas	61
6.3.1	<code>select()</code> : elegir columnas	61
6.3.2	<code>rename()</code> : renombrar columnas	61
6.3.3	<code>mutate()</code> : crear (o modificar) columnas	61
6.3.4	<code>relocate()</code> : reordenar columnas	62
6.3.5	<code>case_when()</code> : recodificación condicional con múltiples criterios	62
6.4	Encadenar transformaciones: el operador <i>pipe</i>	62
6.4.1	Agrupación: <code>group_by()</code>	62
6.4.2	Operar sobre varias columnas a la vez: <code>across()</code>	63
6.5	Reestructurar tablas: <code>tidyr</code>	64
6.5.1	<code>pivot_longer()</code> : de ancho a largo	64
6.5.2	<code>pivot_wider()</code> : de largo a ancho	64
6.6	Verbos de dos tablas: combinar conjuntos de datos	64
6.6.1	<code>left_join()</code> : conservar todas las filas de la tabla izquierda	65
6.6.2	<code>full_join()</code> : conservar todas las filas de ambas tablas	65
6.7	Guardar los resultados	66
7	Visualización de la información	67
7.1	Objetivos	67
7.2	La gramática de gráficos	67
7.3	Gráfico de dispersión	68
7.3.1	Mapeo estético del color	68
7.3.2	Añadir un ajuste: <code>geom_smooth()</code>	68
7.4	Gráfico de barras	69
7.4.1	Proporciones y relleno	69
7.4.2	Coordenadas polares	69
7.5	Facetas: pequeños múltiples	69
7.6	Etiquetas	70
7.6.1	Etiquetas de texto dentro del gráfico	70
7.7	Límites de los ejes: recortar frente a hacer zoom	70
7.8	Temas	71
7.9	Guardar gráficos	71
8	Consultas RDF y SPARQL	73
8.1	Objetivos	73
8.2	Del modelo tabular al modelo de tripletas	73
8.2.1	De un <code>data.frame</code> a tripletas	73
8.2.2	Identificadores URI	74
8.2.3	Tipos de dato (<i>datatype URIs</i>)	74
8.3	El paquete <code>rdflib</code>	75
8.3.1	Prefijos y CURIE	75
8.3.2	Serialización: varios formatos, un mismo grafo	75
8.3.3	De tablas a grafos, y de vuelta	76
8.4	Consultas SPARQL	77
8.4.1	Consultas sobre un grafo en memoria	77
8.4.2	Consultas a <i>endpoints</i> SPARQL remotos	77
9	Prácticas PubMed	81
9.1	Objetivos	81

9.2	Consultas programáticas a PubMed	81
9.2.1	Claves de acceso y <i>E-utilities</i>	81
9.2.2	Construir la consulta	81
9.2.3	Número de resultados antes de descargar	82
9.2.4	Descarga de metadatos	82
9.2.5	De XML a tabla	82
9.3	Análisis bibliométrico con <code>bibliometrix</code>	82
9.3.1	Importar el corpus al formato de <code>bibliometrix</code>	83
9.3.2	Comprobar la integridad de los metadatos	83
9.3.3	Métricas bibliométricas principales	83
9.3.4	Estructura conceptual: análisis de co-palabras	83
9.4	Nubes de palabras a partir de los resúmenes	83
9.4.1	Enfoque con <code>PubMedWordcloud</code>	84
9.4.2	Enfoque <code>tidyverse</code> : <code>tidytext</code> + <code>ggplot2</code>	84
9.5	Gestión de bibliografía en RMarkdown/Quarto	85
9.5.1	Formato BibTeX y estilos CSL	85
9.5.2	Citar en el texto	85
9.5.3	Gestores de bibliografía en la nube	86
9.5.4	Generar la lista de referencias	86
Referencias		87

Listado de Figuras

1.1	Terminal de comandos en Linux.	4
1.2	Terminal de comandos mostrando el directorio de trabajo.	5
1.3	Terminal de comandos mostrando el contenido de una carpeta.	6
1.4	Terminal de comandos mostrando la inicialización de un directorio.	7
1.5	Terminal de comandos mostrando el estado del directorio.	7
1.6	Terminal de comandos mostrando el estado tras <code>git add</code>	8
1.7	Terminal de comandos mostrando la respuesta al <code>commit</code>	9
1.8	Terminal de comandos mostrando el estado del directorio tras el <code>commit</code>	9
1.9	Script original.	10
1.10	Script con la modificación (a realizar por el estudiante).	10
1.11	Terminal mostrando los cambios detectados.	11
1.12	Terminal mostrando el resultado del nuevo <code>commit</code>	12
1.13	Contenido de la carpeta antes del borrado.	12
1.14	Contenido de la carpeta tras el borrado.	13
1.15	Contenido de la carpeta tras <code>git stash</code> : el archivo ha vuelto.	13
1.16	Estado de la carpeta tras el borrado, antes de confirmarlo.	14
1.17	Estado de la carpeta tras confirmar el borrado.	15
1.18	Historial de cambios del repositorio.	15
1.19	Ventana de edición del mensaje de <code>revert</code>	16
1.20	Edición del mensaje de <code>revert</code>	17
1.21	Efecto de <code>revert</code> mostrado en la terminal.	17
1.22	Contenido de la carpeta y estado final del repositorio.	18

Listado de Tablas

1.1	Comandos básicos de Git utilizados en esta práctica.	18
2.1	Comparación entre RMarkdown y Quarto.	28
5.1	Los seis tipos de valor definidos por la especificación JSON (Bray, 2017).	51
5.2	Correspondencia entre los tipos JSON y las estructuras de R usada por <code>jsonlite</code> (Ooms, 2014).	52

Prefacio

Este libro reúne el cuaderno de **prácticas** de la asignatura **Fuentes de Datos Biomédicos y Web Semántica** (#8235) del Grado en Ingeniería de la Salud de la Universidad de Burgos. Es el compañero práctico del libro de teoría de la asignatura: aquí el foco está en el código R, siguiendo el enfoque **tidyverse**  (Wickham et al., 2019) descrito en Wickham et al. (2023), y en cómo aplicarlo para importar, transformar, visualizar y publicar fuentes de datos biomédicas y semánticas.

Estructura del libro

El libro sigue el mismo orden que las carpetas de prácticas del curso (00_ a 08_) y se organiza en nueve capítulos:

0. **Práctica GitHub**: control de versiones con Git desde la línea de comandos (`init`, `status`, `add`, `commit`, `stash`, `log`, `revert`) como base para el trabajo reproducible.
1. **Seminario, GitHub y RMarkdown/Quarto**: sintaxis de los documentos dinámicos (RMarkdown y Quarto), y la estructura del seminario evaluable del curso, que combina texto, código y control de versiones.
2. **Objetos en R**: los tipos de datos y estructuras fundamentales de R (vectores, factores, fechas, matrices, *data frames*, tibbles y listas), siguiendo el modelo de Wickham (2019).
3. **Funciones en R**: control de flujo, iteración (`for`, `while`, `apply`/`tapply`) y definición de funciones propias, con ejemplos biomédicos y de bioinformática.
4. **Importación y exportación de datos**: lectura y escritura de datos estructurados (`.xlsx`, `.csv`) y semiestructurados (**JSON**, XML), con especial atención al trabajo con **JSON** mediante `jsonlite` y `tidyjson`.
5. **Procesamiento y manejo de datos**: manipulación de tablas con `dplyr` y `tidyr` (filtrado, resumen, *pivoting*, combinación de tablas) (Wickham et al., 2025; Wickham, François, et al., 2026).
6. **Visualización de la información**: la gramática de gráficos de `ggplot2` (Wickham, 2016) aplicada a datos biomédicos y ambientales.
7. **Consultas RDF y SPARQL**: representación de datos como tripletas sujeto-predicado-objeto, el paquete `rdflib` y consultas SPARQL a *endpoints* como DBpedia y Wikidata.
8. **Prácticas PubMed**: consulta programática de PubMed con `pubmedR`, análisis bibliométrico con `bibliometrix`, nubes de palabras y gestión de bibliografía en RMarkdown/Quarto.

Cómo usar este libro

Cada capítulo mezcla teoría breve con fragmentos de código en R que ilustran los conceptos presentados. El código se muestra con fines didácticos y **no se ejecuta** al generar este PDF: muchos ejemplos dependen de datos propios de cada estudiante, de servicios web en vivo o de claves de API personales, por lo que se recomienda ejecutar el código directamente en RStudio, siguiendo el propio material del curso disponible en el repositorio del curso.

Al inicio de cada capítulo se indican los paquetes de R necesarios. La instalación conjunta de todos los paquetes del curso puede hacerse ejecutando el script `Instalar_Paquetes_Curso_8235.R` disponible en la raíz del repositorio de prácticas.

Licencia

Autor: Antonio Canepa Oneto

Área: Lenguajes y Sistemas Informáticos

Departamento: Ingeniería Informática

Escuela Politécnica Superior, Universidad de Burgos



Esta obra está bajo una licencia Creative Commons *Reconocimiento-NoComercial-CompartirIgual 3.0 Unported*. No se permite un uso comercial de esta obra ni de las posibles obras derivadas; la distribución de las mismas debe hacerse con una licencia igual a la que regula esta obra original. Licencia disponible en creativecommons.org/licenses/by-nc-sa/3.0.

1 Práctica GitHub

1.1. Objetivos

En esta primera práctica no escribiremos ni una línea de código en R. Antes de manipular datos biomédicos necesitamos una herramienta que garantice que ese trabajo sea **reproducible** y **recuperable**: un sistema de control de versiones. Trabajaremos con **Git** desde la línea de comandos, incluso sin necesidad de estar conectados a un repositorio remoto como GitHub, para entender su lógica interna antes de automatizarla desde RStudio.

Al finalizar esta práctica seréis capaces de:

- Diferenciar un directorio de trabajo ordinario de un **repositorio** vigilado por Git.
- Registrar el histórico de cambios de un proyecto mediante *commits*.
- Recuperar archivos borrados, tanto si el borrado fue registrado (*committed*) como si no.
- Interpretar la salida de `git status` y `git log`, las dos órdenes que más se ejecutan en el día a día.

1.2. ¿Por qué control de versiones en ciencia de datos?

Un análisis de datos biomédicos rara vez se escribe de una sola vez: se corrige un filtro, se añade una variable, se cambia el criterio de exclusión de pacientes, etc. Sin un sistema de control de versiones, ese historial de decisiones se pierde o queda disperso en archivos con nombres como `analisis_final_v2_definitivo.R`. Git resuelve este problema guardando, de forma eficiente, el estado completo de un proyecto en cada punto en el que decidimos “fijarlo” (un *commit*), permitiendo:

- **Reproducibilidad**: cualquier resultado puede vincularse al identificador exacto del código que lo generó.
- **Reversibilidad**: un error de ayer no destruye el trabajo de hoy; siempre se puede volver a un estado anterior.
- **Colaboración**: varias personas pueden trabajar sobre el mismo proyecto sin sobrescribirse (aspecto que veremos con más detalle al usar repositorios remotos en GitHub, en el Capítulo 2).

A nivel conceptual, Git no almacena una lista de diferencias entre versiones (como hacían sistemas más antiguos), sino una serie de “fotografías” (*snapshots*) del proyecto completo; cada *commit* apunta a la fotografía anterior, formando una cadena histórica (Chacon & Straub, 2014). Esta práctica se limita al uso **local** de Git; la integración con GitHub y RStudio se retoma en el Capítulo 2.

1.3. Preparación del directorio de trabajo

Para este ejercicio necesitaréis únicamente dos archivos: `Analysis.R` y `Datos.txt`. Cread una carpeta de trabajo que contenga solo estos dos archivos: ningún otro fichero, ni siquiera oculto, debe interferir con el ejercicio.

A continuación abriremos la línea de comandos del sistema operativo (la **terminal**). En Linux, la terminal luce como en la Figura 1.1.

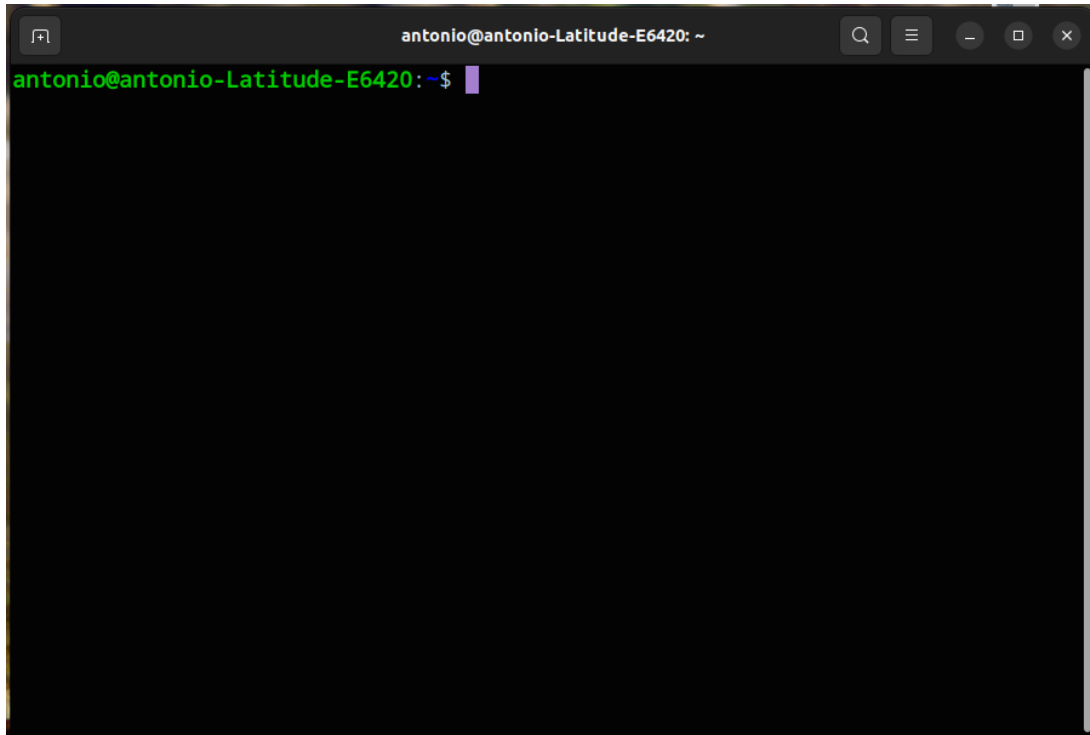


Figura 1.1: Terminal de comandos en Linux.

El directorio en el que “escucha” la terminal al abrirse no tiene por qué coincidir con el directorio donde guardasteis los archivos. Para saber en qué carpeta está situada la terminal en cada momento, existen dos comandos equivalentes:

- `pwd` (*print working directory*): en Linux y macOS.
- `cd` (sin argumentos): en Windows, muestra el directorio actual.

El resultado es similar al de la Figura 1.2.

Para desplazarnos hasta la carpeta que contiene `Analysis.R` y `Datos.txt` disponemos de dos estrategias:

1. Abrir la terminal directamente desde la carpeta:

- En *Linux*, desde el explorador de archivos, botón derecho sobre la carpeta → “abrir en la terminal”.
- En *Windows*, botón derecho sobre la carpeta → “abrir ventana de comandos aquí” (o *Git Bash here*, si tenéis instalado Git para Windows).



Figura 1.2: Terminal de comandos mostrando el directorio de trabajo.

- En *macOS*, la opción más directa es arrastrar la carpeta sobre el icono de la Terminal, lo que completa la ruta automáticamente.
2. **Navegar con `cd` (change directory)** desde una terminal ya abierta. Es la aproximación que seguiremos en el curso, porque el comando es idéntico en los tres sistemas operativos, aunque la ruta concreta dependa de dónde guarde cada estudiante sus archivos:

Listado 1.1 terminal

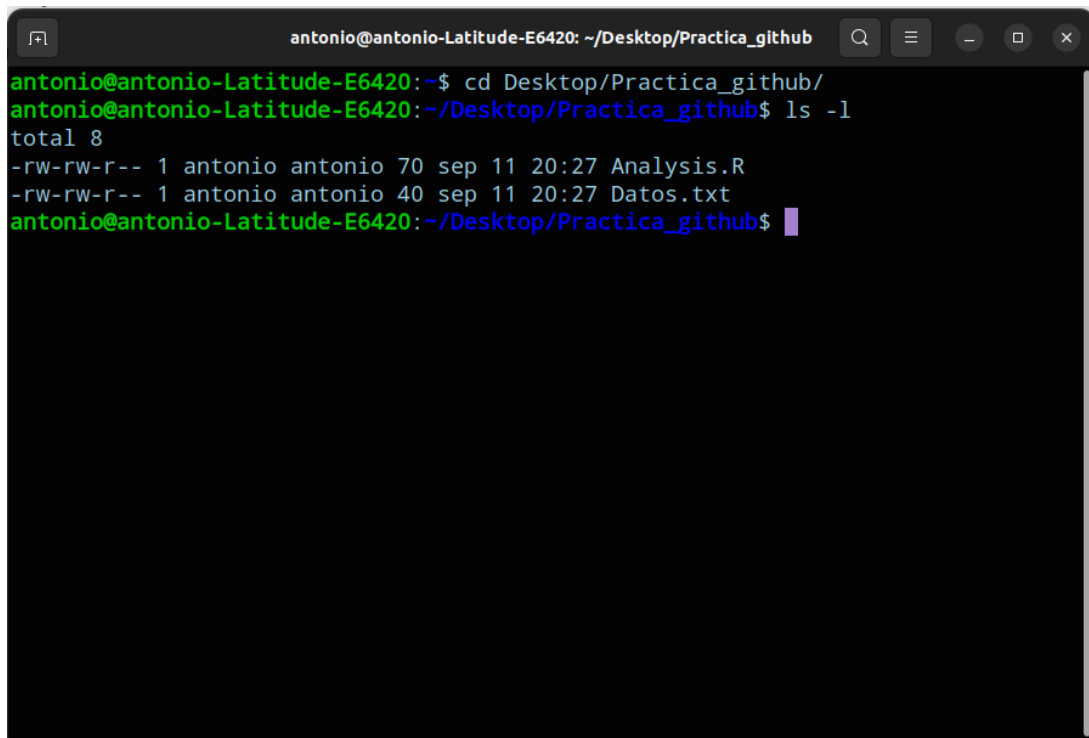
```
1  cd Desktop/Practica_github/
```

Para confirmar que estamos en el lugar correcto, repetimos `pwd/cd` o listamos el contenido de la carpeta:

Listado 1.2 terminal

```
1  ls -l    # Linux / macOS
2  dir      # Windows
```

Con los archivos correctos delante, pasamos a los comandos básicos de Git.

A terminal window titled 'antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github'. The prompt is 'antonio@antonio-Latitude-E6420:~\$'. The user enters 'cd Desktop/Practica_github/'. The prompt changes to 'antonio@antonio-Latitude-E6420:~/Desktop/Practica_github\$'. The user enters 'ls -l'. The output shows 'total 8', followed by two lines: '-rw-rw-r-- 1 antonio antonio 70 sep 11 20:27 Analysis.R' and '-rw-rw-r-- 1 antonio antonio 40 sep 11 20:27 Datos.txt'. The prompt returns to 'antonio@antonio-Latitude-E6420:~/Desktop/Practica_github\$' with a cursor.

```
antonio@antonio-Latitude-E6420:~$ cd Desktop/Practica_github/
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 8
-rw-rw-r-- 1 antonio antonio 70 sep 11 20:27 Analysis.R
-rw-rw-r-- 1 antonio antonio 40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

Figura 1.3: Terminal de comandos mostrando el contenido de una carpeta.

1.4. Comandos básicos de Git

1.4.1. Inicializar un repositorio

Si la carpeta actual no está vigilada por Git, debemos indicárselo explícitamente: esto es **inicializar** un repositorio.

Este comando crea una subcarpeta oculta `.git/` que almacena todo el historial y la configuración del repositorio. Borrar esa carpeta (algo que nunca deberíamos hacer sin querer) equivale a perder todo el historial de versiones, aunque los archivos de trabajo actuales permanezcan intactos.

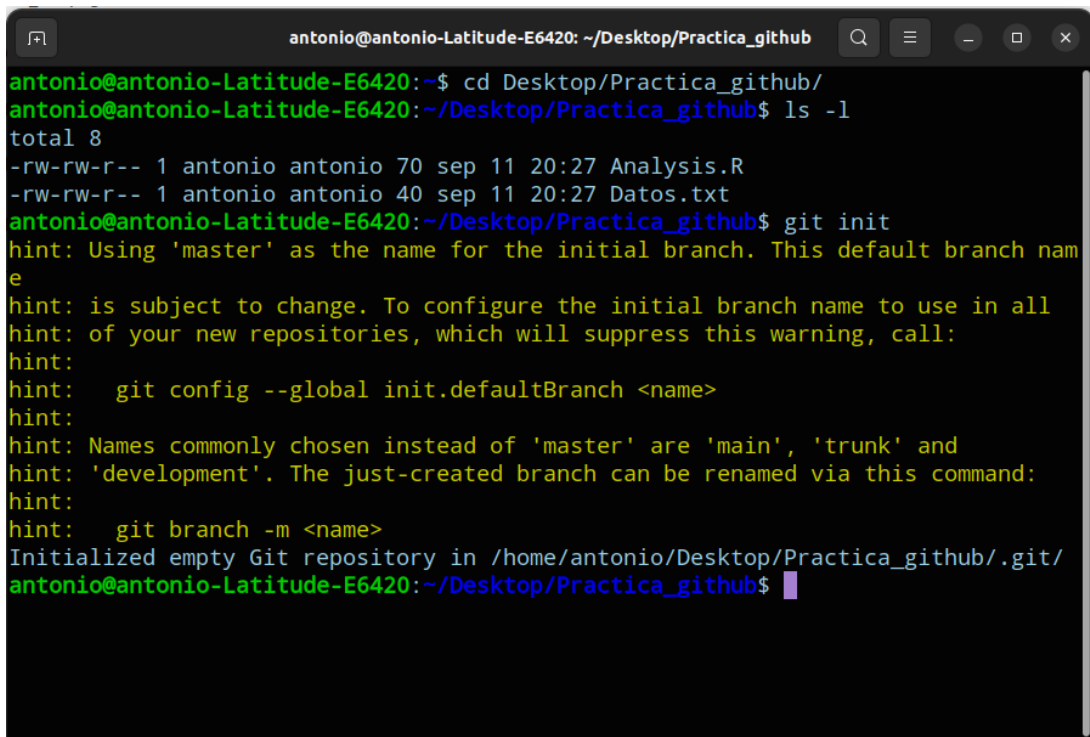
1.4.1.1. Verificar el estado del repositorio

El comando que más se ejecuta en el flujo de trabajo con Git es, con diferencia, `git status`:

`git status` nos informa de tres posibles estados por archivo: sin rastrear (*untracked*), modificado y no preparado (*not staged*), o preparado para el próximo *commit* (*staged*). En la Figura 1.5 vemos que los archivos aparecen como *untracked*: existen en la carpeta, pero Git aún no los vigila.

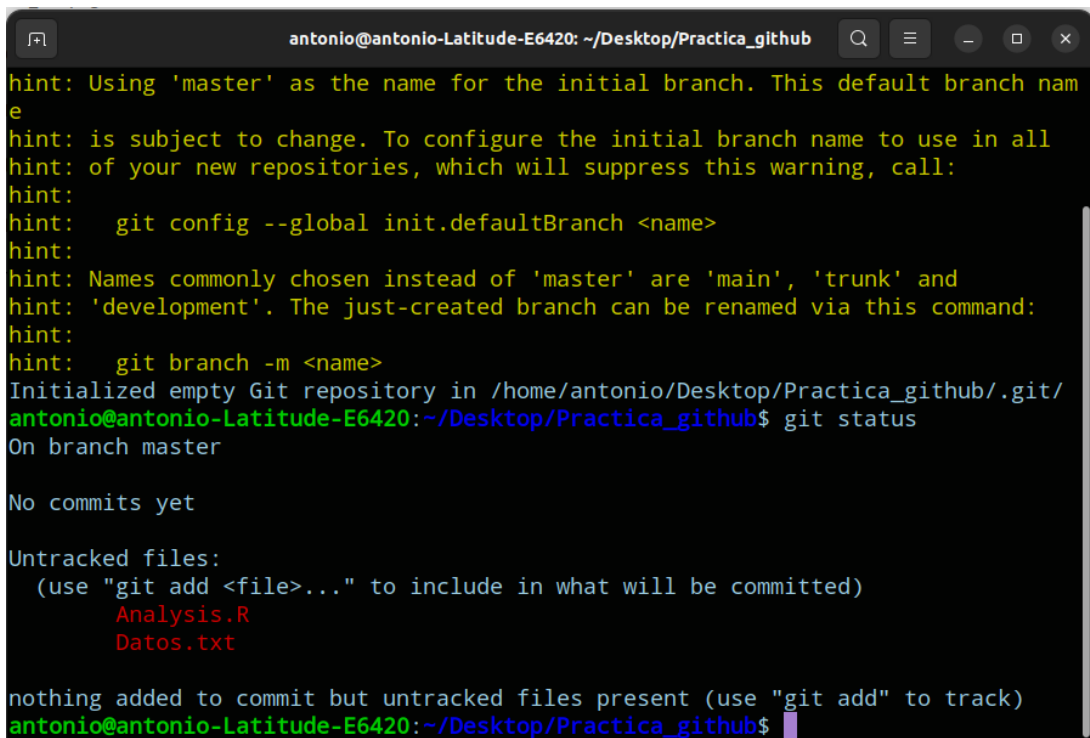
1.4.1.2. Rastreo de archivos

Para incorporar archivos al área de *staging* usamos `git add <archivo>`. Como acordamos vigilar **todo** el directorio, sustituimos el nombre de cada archivo por el comodín `.` (punto), que significa *todos los archivos del directorio actual*:



```
antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~$ cd Desktop/Practica_github/
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 8
-rw-rw-r-- 1 antonio antonio 70 sep 11 20:27 Analysis.R
-rw-rw-r-- 1 antonio antonio 40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/antonio/Desktop/Practica_github/.git/
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

Figura 1.4: Terminal de comandos mostrando la inicialización de un directorio.



```
antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/antonio/Desktop/Practica_github/.git/
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Analysis.R
    Datos.txt

nothing added to commit but untracked files present (use "git add" to track)
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

Figura 1.5: Terminal de comandos mostrando el estado del directorio.

Si repetimos `git status`, los archivos ya no aparecen como *untracked*, sino listos para el próximo *commit* (Figura 1.6).

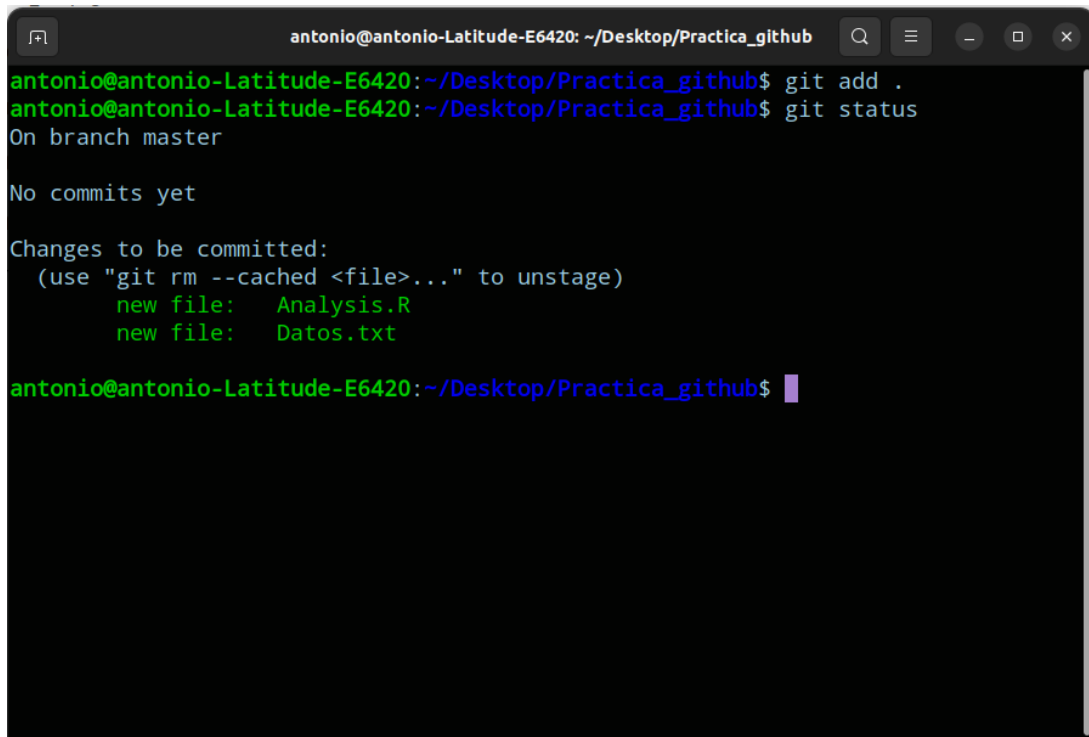
A terminal window titled 'antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github'. The prompt is 'antonio@antonio-Latitude-E6420:~/Desktop/Practica_github\$'. The user enters 'git add .' and then 'git status'. The output shows 'On branch master', 'No commits yet', and 'Changes to be committed:' followed by '(use "git rm --cached <file>..." to unstage)', 'new file: Analysis.R', and 'new file: Datos.txt'. The prompt returns to 'antonio@antonio-Latitude-E6420:~/Desktop/Practica_github\$'.

Figura 1.6: Terminal de comandos mostrando el estado tras `git add ..`

1.4.1.3. Registro de cambios: los *commits*

Rastrear un archivo (`git add`) no equivale a fijar su estado en el historial. Para ello necesitamos crear un **commit**, que es un punto de guardado permanente identificado por un mensaje descriptivo:

Si volvemos a preguntar por el estado del repositorio...

...obtenemos el mensaje que **siempre queremos ver**:

```
On branch master
nothing to commit, working tree clean
```

Este mensaje confirma que el estado actual de los archivos coincide exactamente con el último *commit* registrado: no hay cambios pendientes de rastrear ni de confirmar.

1.5. Modificación de ficheros

1.5.1. Modificar archivos existentes

Abrimos `Analysis.R` y añadimos un comentario en la primera línea, pasando del script original (Figura 1.9) al modificado (Figura 1.10).

```
antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git add .
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Analysis.R
        new file:   Datos.txt

antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git commit -m "inicio
del proyecto"
[master (root-commit) 262ebf4] inicio del proyecto
 2 files changed, 5 insertions(+)
 create mode 100644 Analysis.R
 create mode 100644 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

Figura 1.7: Terminal de comandos mostrando la respuesta al commit.

```
antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git add .
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Analysis.R
        new file:   Datos.txt

antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git commit -m "inicio
del proyecto"
[master (root-commit) 262ebf4] inicio del proyecto
 2 files changed, 5 insertions(+)
 create mode 100644 Analysis.R
 create mode 100644 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
nothing to commit, working tree clean
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

Figura 1.8: Terminal de comandos mostrando el estado del directorio tras el commit.

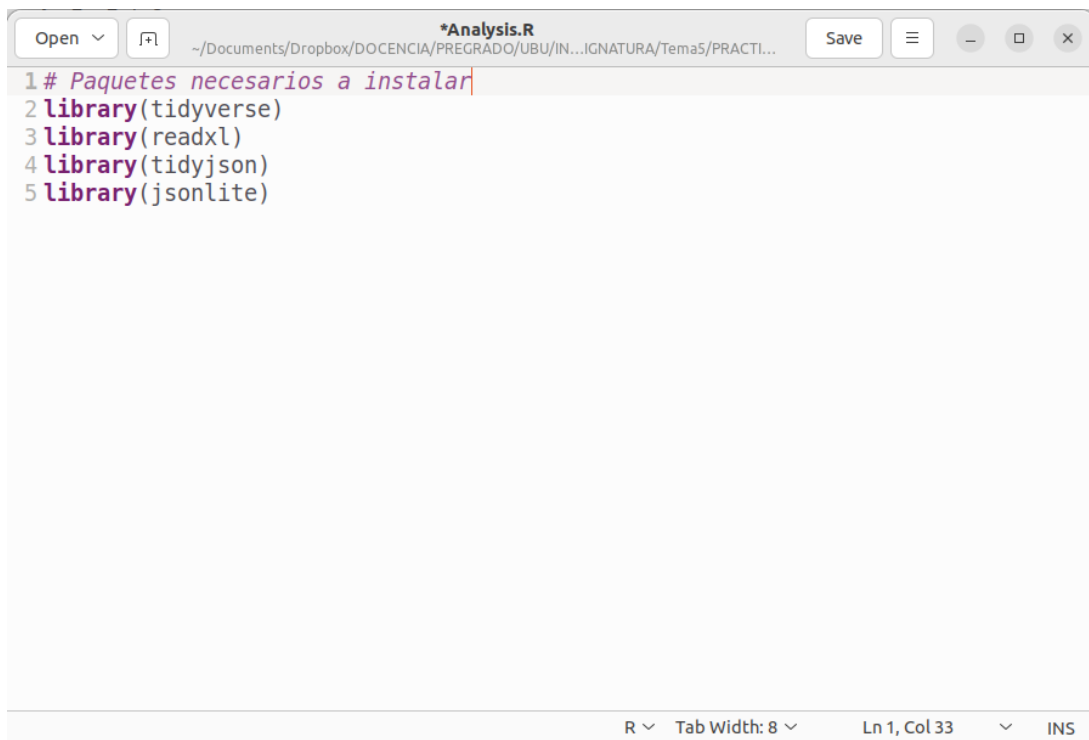
1 Práctica GitHub



The screenshot shows an R script editor window titled "Analysis.R". The file path is "~/Documents/Dropbox/DOCENCIA/PREGRADO/UBU/IN...IGNATURA/Tema5/PRACTI...". The script contains four lines of code: `library(tidyverse)`, `library(readxl)`, `library(tidyjson)`, and `library(jsonlite)`. The status bar at the bottom indicates "R", "Tab Width: 8", "Ln 1, Col 1", and "INS".

```
1 library(tidyverse)
2 library(readxl)
3 library(tidyjson)
4 library(jsonlite)
```

Figura 1.9: Script original.



The screenshot shows the same R script editor window, but the script has been modified. The first line is a comment: `# Paquetes necesarios a instalar`. The subsequent lines are the same four library calls as in the original script: `library(tidyverse)`, `library(readxl)`, `library(tidyjson)`, and `library(jsonlite)`. The status bar at the bottom indicates "R", "Tab Width: 8", "Ln 1, Col 33", and "INS".

```
1 # Paquetes necesarios a instalar
2 library(tidyverse)
3 library(readxl)
4 library(tidyjson)
5 library(jsonlite)
```

Figura 1.10: Script con la modificación (a realizar por el estudiante).

Al ejecutar de nuevo `git status`, Git detecta el cambio y nos indica los pasos a seguir (Figura 1.11):

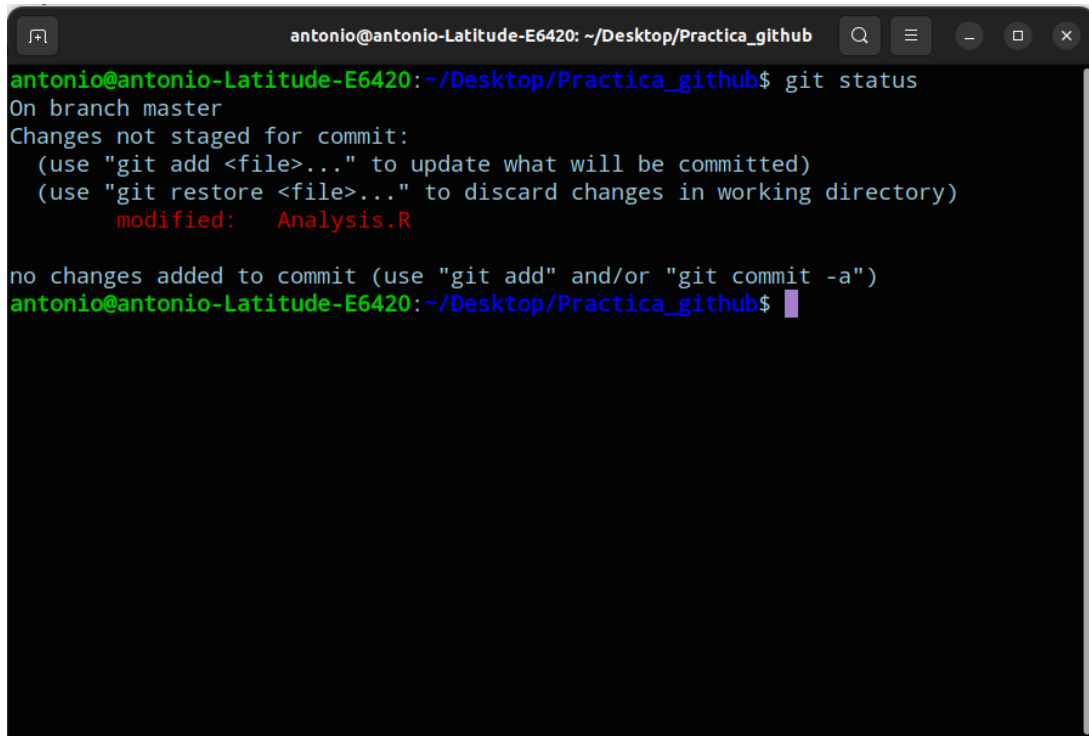
A terminal window titled 'antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github'. The prompt is 'antonio@antonio-Latitude-E6420:~/Desktop/Practica_github\$'. The command 'git status' has been executed. The output shows 'On branch master' and 'Changes not staged for commit:'. It lists 'modified: Analysis.R' in red. It also provides instructions on how to stage changes with 'git add' or discard them with 'git restore'. The prompt returns to 'antonio@antonio-Latitude-E6420:~/Desktop/Practica_github\$'.

Figura 1.11: Terminal mostrando los cambios detectados.

Como en el paso anterior, rastreamos el cambio y lo registramos con un nuevo *commit*:

1.5.2. Borrar archivos y recuperarlos

Accidentalmente borrar un archivo es uno de los errores más comunes en cualquier proyecto. Gracias a Git podemos “volver sobre nuestros pasos”, pero **el procedimiento de recuperación depende de si el borrado llegó a registrarse en un *commit*.**

1.5.2.1. Borrado sin *commit*

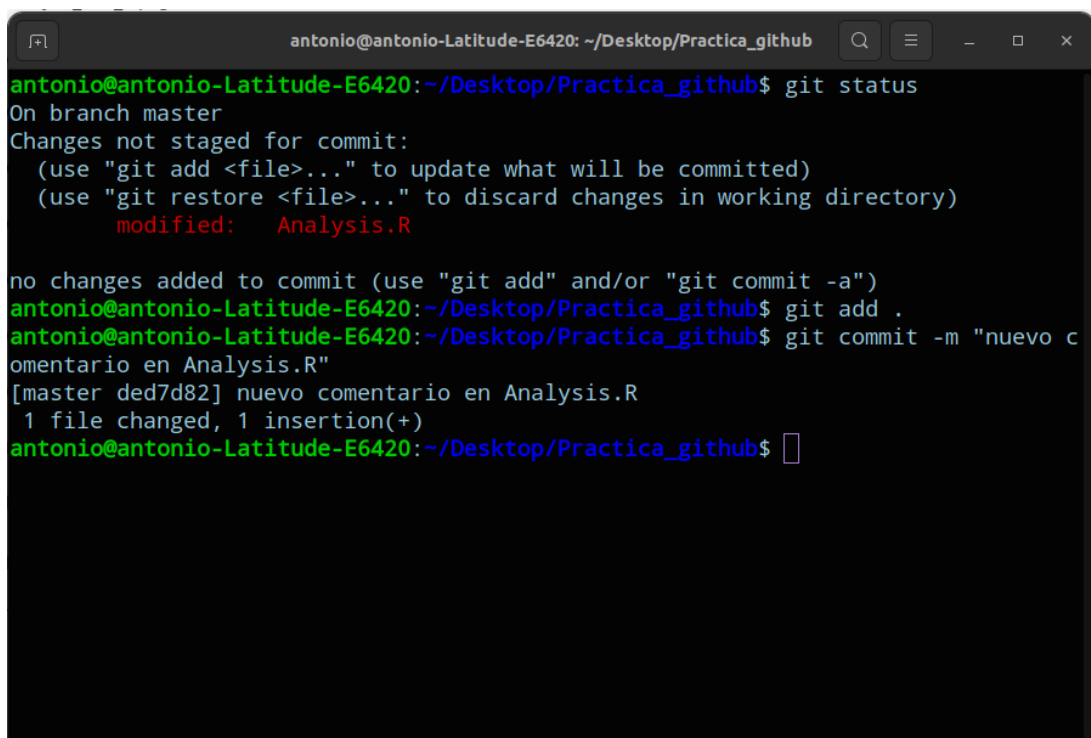
Este es el caso más sencillo: como el borrado aún no se ha confirmado, basta con un equivalente a “deshacer” (*undo*). En Git, ese mecanismo es `git stash`, que guarda temporalmente los cambios no confirmados y restaura el último estado registrado.

Listamos el contenido de la carpeta:

Borramos `Analysis.R` (`rm` en Linux/macOS, `del` en Windows) y comprobamos que ha desaparecido:

Como este cambio **no está confirmado**, `git stash` es suficiente para recuperar el archivo:

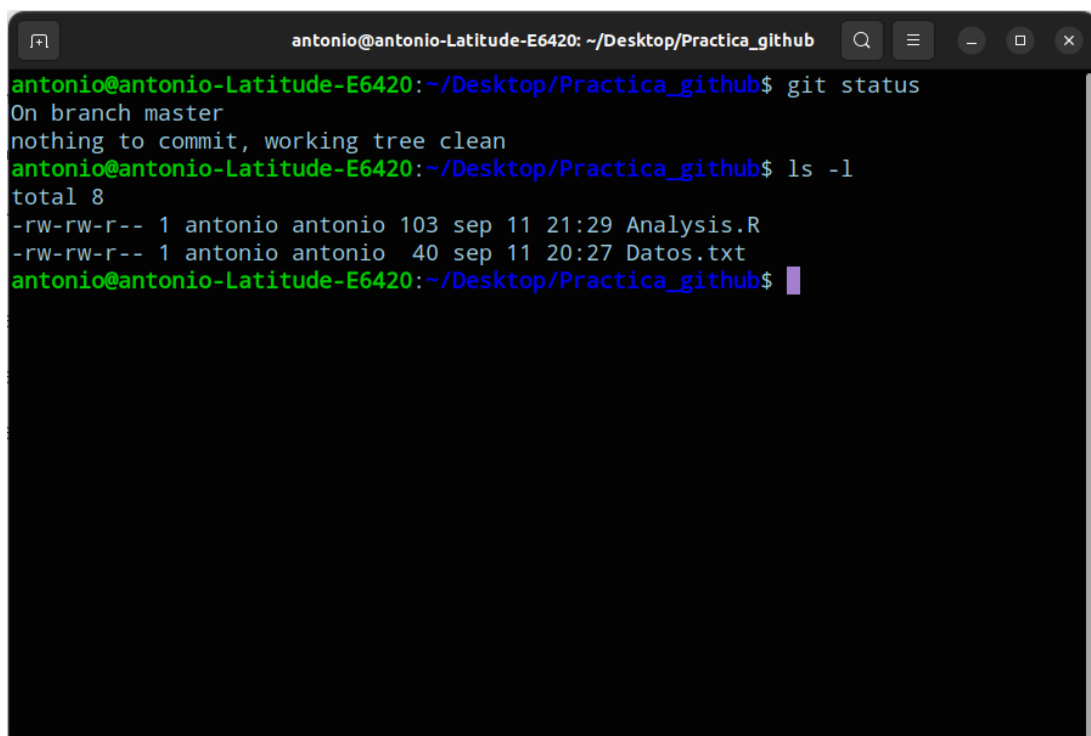
1 Práctica GitHub



```
antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Analysis.R

no changes added to commit (use "git add" and/or "git commit -a")
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git add .
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git commit -m "nuevo c
omentario en Analysis.R"
[master ded7d82] nuevo comentario en Analysis.R
 1 file changed, 1 insertion(+)
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

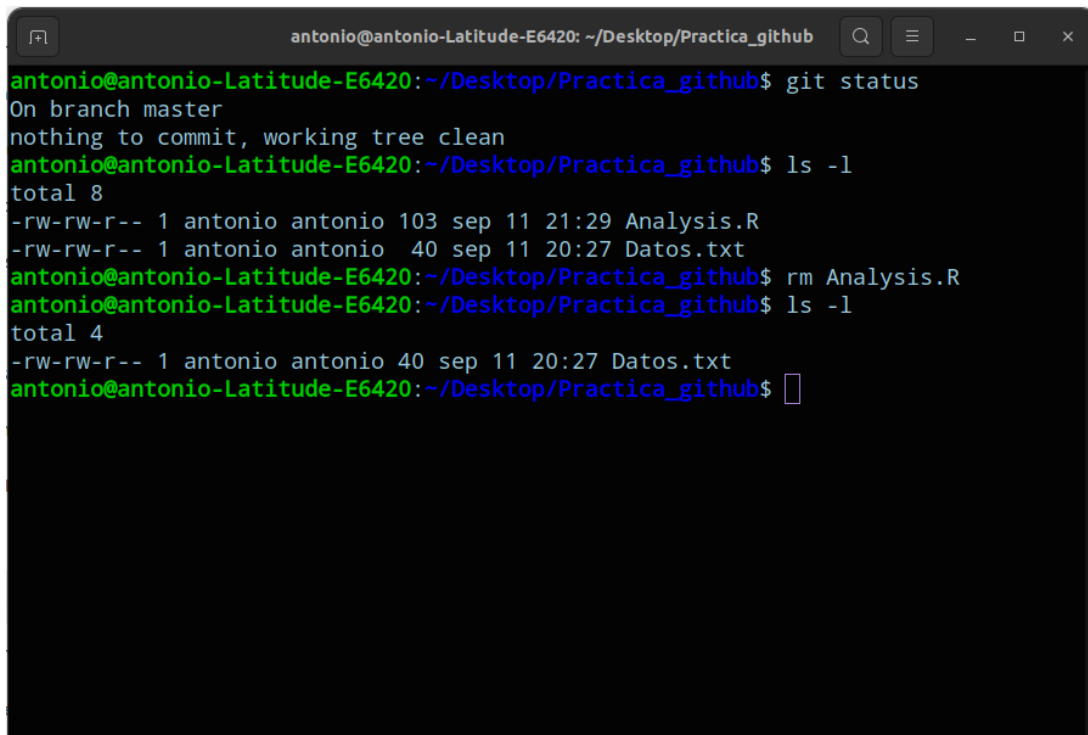
Figura 1.12: Terminal mostrando el resultado del nuevo commit.



```
antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
nothing to commit, working tree clean
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 8
-rw-rw-r-- 1 antonio antonio 103 sep 11 21:29 Analysis.R
-rw-rw-r-- 1 antonio antonio  40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

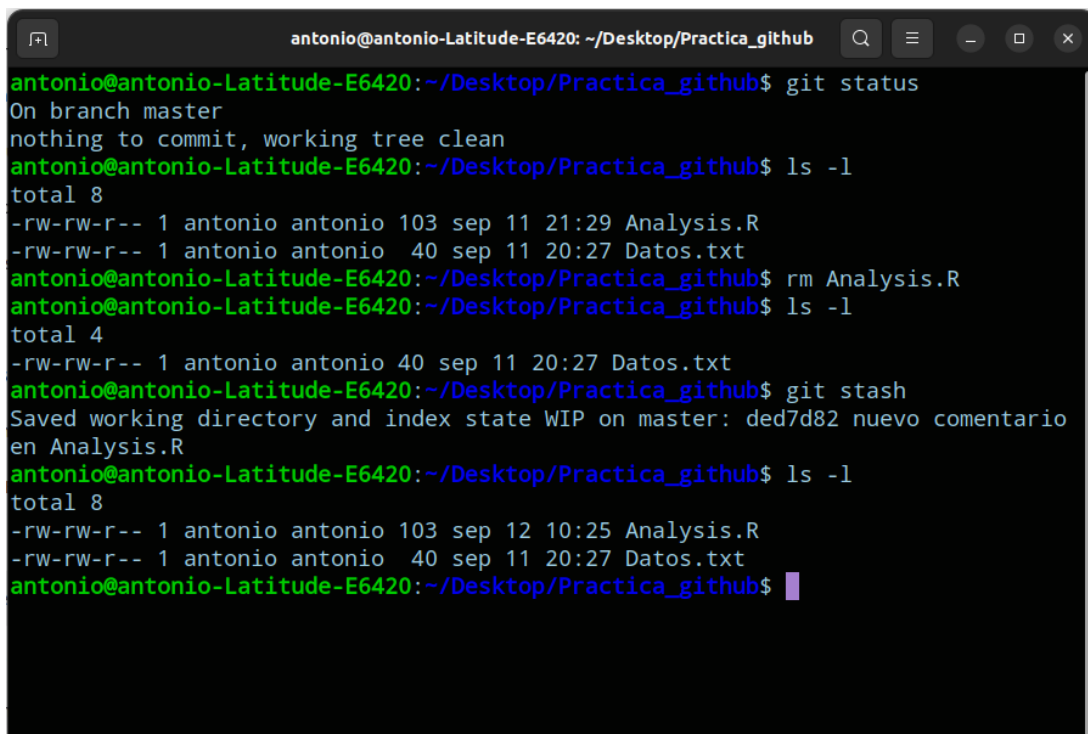
Figura 1.13: Contenido de la carpeta antes del borrado.

1 Práctica GitHub

A terminal window titled 'antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github'. The user runs 'git status', which shows 'On branch master' and 'nothing to commit, working tree clean'. Then they run 'ls -l', showing two files: 'Analysis.R' (103 bytes, Sep 11 21:29) and 'Datos.txt' (40 bytes, Sep 11 20:27). Next, they run 'rm Analysis.R'. Finally, they run 'ls -l' again, showing only 'Datos.txt' (40 bytes, Sep 11 20:27).

```
antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
nothing to commit, working tree clean
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 8
-rw-rw-r-- 1 antonio antonio 103 sep 11 21:29 Analysis.R
-rw-rw-r-- 1 antonio antonio  40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ rm Analysis.R
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 4
-rw-rw-r-- 1 antonio antonio 40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

Figura 1.14: Contenido de la carpeta tras el borrado.

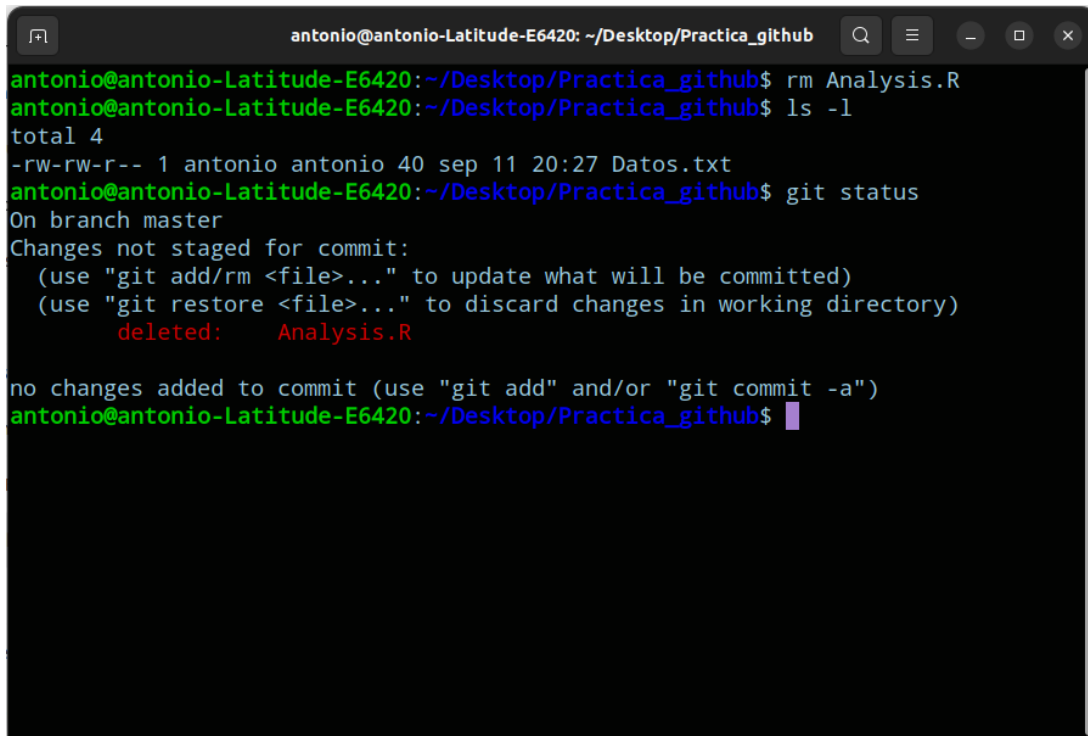
A terminal window titled 'antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github'. The user runs 'git status' (clean), 'ls -l' (shows both files), 'rm Analysis.R', and 'ls -l' (shows only Datos.txt). Then they run 'git stash', which outputs 'Saved working directory and index state WIP on master: ded7d82 nuevo comentario en Analysis.R'. Finally, they run 'ls -l', which now shows both 'Analysis.R' (103 bytes, Sep 12 10:25) and 'Datos.txt' (40 bytes, Sep 11 20:27).

```
antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
nothing to commit, working tree clean
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 8
-rw-rw-r-- 1 antonio antonio 103 sep 11 21:29 Analysis.R
-rw-rw-r-- 1 antonio antonio  40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ rm Analysis.R
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 4
-rw-rw-r-- 1 antonio antonio 40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git stash
Saved working directory and index state WIP on master: ded7d82 nuevo comentario
en Analysis.R
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 8
-rw-rw-r-- 1 antonio antonio 103 sep 12 10:25 Analysis.R
-rw-rw-r-- 1 antonio antonio  40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$
```

Figura 1.15: Contenido de la carpeta tras git stash: el archivo ha vuelto.

1.5.2.2. Borrado con *commit*

Esta situación es algo más compleja, pero también más habitual: el borrado se registra en el historial, por lo que `git stash` deja de ser una opción válida.



```

antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ rm Analysis.R
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 4
-rw-rw-r-- 1 antonio antonio 40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    Analysis.R

no changes added to commit (use "git add" and/or "git commit -a")
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$

```

Figura 1.16: Estado de la carpeta tras el borrado, antes de confirmarlo.

Rastreamos y confirmamos el borrado como cualquier otro cambio:

El archivo `Analysis.R` ya no está presente y, además, su ausencia ha sido **rastreada** y **confirmada**: `git stash` ya no sirve de nada en este punto. Sin embargo, precisamente por haber quedado registrado en el historial, sí podemos recuperarlo revirtiendo el *commit* que lo eliminó.

1.5.2.2.1. Consultar el historial: `git log`

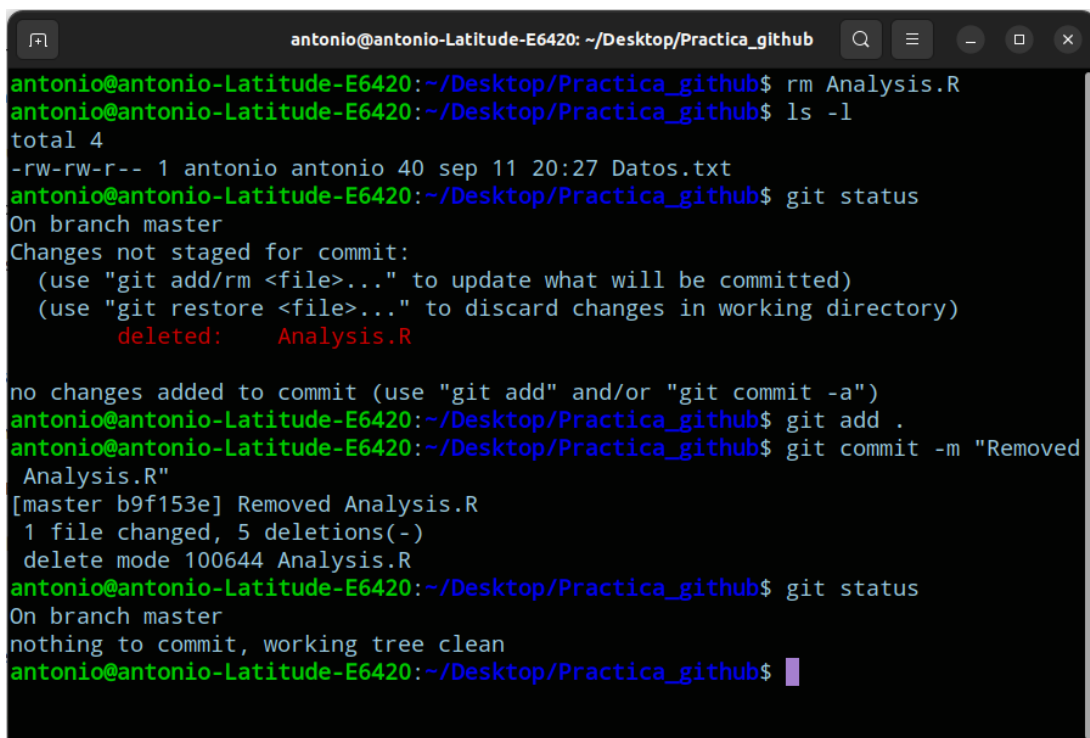
Para localizar el *commit* que eliminó el archivo consultamos el historial completo del repositorio:

En este historial, los cambios más recientes aparecen primero (más cerca de la línea de comandos). Cada entrada muestra el **identificador único del commit** (*Commit Unique Identifier*, CUI): un código alfanumérico de 40 caracteres (un *hash* SHA-1) situado justo a la derecha de la palabra *commit*. En nuestro ejemplo, el *commit* que eliminó el archivo tiene el siguiente CUI:

- CUI: b9f153e1d344cb2d9c4c03ab3589299d0100e972

1.5.2.2.2. Revertir un *commit*

Para deshacer los efectos de un *commit* concreto sin reescribir el historial (una operación segura incluso en repositorios compartidos) usamos `git revert <CUI>`. Basta con los primeros dígitos del identificador para que Git lo reconozca sin ambigüedad:



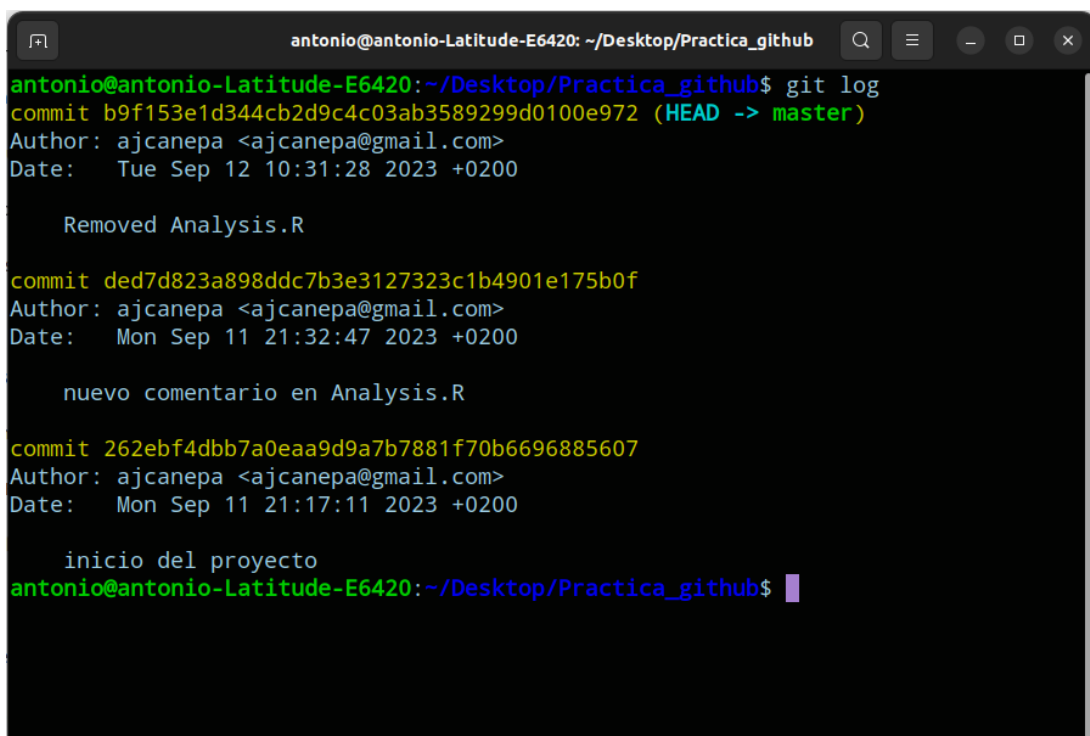
```

antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ rm Analysis.R
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 4
-rw-rw-r-- 1 antonio antonio 40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    Analysis.R

no changes added to commit (use "git add" and/or "git commit -a")
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git add .
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git commit -m "Removed
Analysis.R"
[master b9f153e] Removed Analysis.R
1 file changed, 5 deletions(-)
delete mode 100644 Analysis.R
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
nothing to commit, working tree clean
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$

```

Figura 1.17: Estado de la carpeta tras confirmar el borrado.



```

antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git log
commit b9f153e1d344cb2d9c4c03ab3589299d0100e972 (HEAD -> master)
Author: ajcanepa <ajcanepa@gmail.com>
Date:   Tue Sep 12 10:31:28 2023 +0200

    Removed Analysis.R

commit ded7d823a898ddc7b3e3127323c1b4901e175b0f
Author: ajcanepa <ajcanepa@gmail.com>
Date:   Mon Sep 11 21:32:47 2023 +0200

    nuevo comentario en Analysis.R

commit 262ebf4dbb7a0eaa9d9a7b7881f70b6696885607
Author: ajcanepa <ajcanepa@gmail.com>
Date:   Mon Sep 11 21:17:11 2023 +0200

    inicio del proyecto
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$

```

Figura 1.18: Historial de cambios del repositorio.

Al ejecutar `git revert`, Git abre un editor de texto (por defecto, `vim`) para que redactemos el mensaje del nuevo *commit* que anula al anterior.

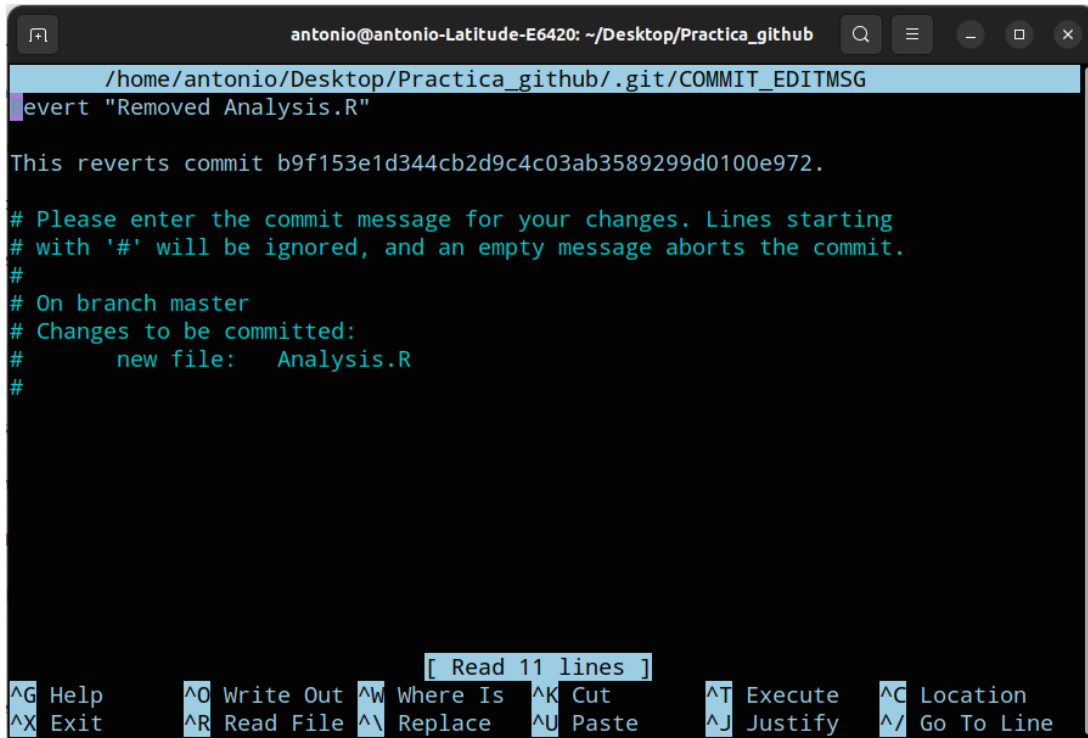


Figura 1.19: Ventana de edición del mensaje de `revert`.

Redactamos el mensaje (parte del texto ya viene generado automáticamente por Git):

En Linux/macOS, con `vim` abierto, `^` equivale a `Ctrl` (`Cmd` en Mac). Para guardar y salir pulsamos `Ctrl + X` (según la versión de `vim`, puede requerir antes `Esc` seguido de `:wq`):

i Particularidad en Windows

En Windows, el editor `vim` no muestra las combinaciones `^G/^X` en la parte inferior. Para escribir, pulsad primero la tecla `i` (modo inserción; deja de verse el texto `-INSERT-` cuando termináis). Para salir y guardar, pulsad `Esc` y a continuación escribid:

Listado 1.15 `vim`

```
1  :wq
```

seguido de `Enter`.

Por último, comprobamos que el archivo ha vuelto y que el árbol de trabajo está limpio:

```

antonio@antonio-Latitude-E6420: ~/Desktop/Practica_github
/home/antonio/Desktop/Practica_github/.git/COMMIT_EDITMSG *
Revert "Removed Analysis.R" porque me equivoqué..jejeje

This reverts commit b9f153e1d344cb2d9c4c03ab3589299d0100e972.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
# Changes to be committed:
#   new file:   Analysis.R
#

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^_ Go To Line

```

Figura 1.20: Edición del mensaje de revert.

```

antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git log
commit b9f153e1d344cb2d9c4c03ab3589299d0100e972 (HEAD -> master)
Author: ajcanepa <ajcanepa@gmail.com>
Date: Tue Sep 12 10:31:28 2023 +0200

    Removed Analysis.R

commit ded7d823a898ddc7b3e3127323c1b4901e175b0f
Author: ajcanepa <ajcanepa@gmail.com>
Date: Mon Sep 11 21:32:47 2023 +0200

    nuevo comentario en Analysis.R

commit 262ebf4dbb7a0eaa9d9a7b7881f70b6696885607
Author: ajcanepa <ajcanepa@gmail.com>
Date: Mon Sep 11 21:17:11 2023 +0200

    inicio del proyecto
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git revert b9f153e1d34
4cb2d9c4c03ab3589299d0100e972
[master fee8494] Revert "Removed Analysis.R" porque me equivoqué..jejeje
1 file changed, 5 insertions(+)
 create mode 100644 Analysis.R
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$

```

Figura 1.21: Efecto de revert mostrado en la terminal.


```

commit ded7d823a898ddc7b3e3127323c1b4901e175b0f
Author: ajcanepa <ajcanepa@gmail.com>
Date:   Mon Sep 11 21:32:47 2023 +0200

    nuevo comentario en Analysis.R

commit 262ebf4dbb7a0eaa9d9a7b7881f70b6696885607
Author: ajcanepa <ajcanepa@gmail.com>
Date:   Mon Sep 11 21:17:11 2023 +0200

    inicio del proyecto
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git revert b9f153e1d34
4cb2d9c4c03ab3589299d0100e972
[master fee8494] Revert "Removed Analysis.R" porque me equivoqué..jejeje
 1 file changed, 5 insertions(+)
 create mode 100644 Analysis.R
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ ls -l
total 8
-rw-rw-r-- 1 antonio antonio 103 sep 12 10:43 Analysis.R
-rw-rw-r-- 1 antonio antonio  40 sep 11 20:27 Datos.txt
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$ git status
On branch master
nothing to commit, working tree clean
antonio@antonio-Latitude-E6420:~/Desktop/Practica_github$

```

Figura 1.22: Contenido de la carpeta y estado final del repositorio.

1.6. Resumen de comandos

La Tabla 1.1 recoge los comandos de Git utilizados en esta práctica, que forman el núcleo mínimo necesario para trabajar de forma reproducible durante el resto del curso.

Tabla 1.1: Comandos básicos de Git utilizados en esta práctica.

Comando	Efecto
<code>git init</code>	Inicializa un repositorio en la carpeta actual
<code>git status</code>	Muestra el estado de los archivos (<i>untracked</i> / <i>staged</i> / limpio)
<code>git add <archivo> / git add .</code>	Rastrea un archivo (o todos) para el próximo <i>commit</i>
<code>git commit -m "mensaje"</code>	Registra permanentemente el estado rastreado
<code>git stash</code>	Restaura el último estado confirmado (solo cambios sin <i>commit</i>)
<code>git log</code>	Muestra el historial de <i>commits</i> y sus identificadores (CUI)
<code>git revert <CUI></code>	Crea un nuevo <i>commit</i> que deshace los efectos de otro anterior

Estos comandos son la base sobre la que trabajaremos en el Capítulo 2, donde conectaremos un repositorio local con GitHub para publicar y compartir el trabajo del seminario evaluable del curso.

1.7. Licencia

Autor: Antonio Canepa Oneto

Área: Lenguajes y Sistemas Informáticos

Departamento: Ingeniería Informática

Escuela Politécnica Superior, Universidad de Burgos



Esta obra está bajo una licencia Creative Commons *Reconocimiento-NoComercial-CompartirIgual 3.0 Unported*. No se permite un uso comercial de esta obra ni de las posibles obras derivadas; la distribución de las mismas debe hacerse con una licencia igual a la que regula esta obra original. Licencia disponible en creativecommons.org/licenses/by-nc-sa/3.0.

Listado 1.3 terminal

Listado 1.4 terminal

```
1  git status
```

Listado 1.5 terminal

Listado 1.6 terminal

```
1  git commit -m "inicio del proyecto"
```

Listado 1.7 terminal

```
1  git add .
2  git commit -m "nuevo comentario en Analysis.R"
```

Listado 1.8 terminal

```
1  ls -l      # dir en Windows
```

Listado 1.9 terminal

```
1  rm Analysis.R    # del "Analysis.R" en Windows
2  ls -l
```

Listado 1.10 terminal

```
1  git stash
```

Listado 1.11 terminal

Listado 1.12 terminal

```
1  git add .
2  git commit -m "Removed Analysis.R"
3  git status
```

Listado 1.13 terminal

```
1  git log
```

Listado 1.14 terminal

Listado 1.16 terminal

```
1  ls -l
2  git status
```

2 Seminario, GitHub y RMarkdown/Quarto

2.1. Objetivos

El Capítulo 1 presentó Git como herramienta local de control de versiones. En esta práctica damos dos pasos más: conectamos ese repositorio local con un repositorio remoto en **GitHub**, y aprendemos a redactar **documentos dinámicos** (RMarkdown y Quarto) que combinan texto, código y resultados en un único archivo reproducible. Ambas herramientas son la base del **seminario evaluable** del curso, cuya estructura se detalla al final del capítulo.

2.2. Documentos dinámicos: RMarkdown y Quarto

Un informe científico convencional separa el análisis (script de R) del texto que lo describe (documento de Word o PDF). Esta separación es una fuente constante de errores: basta con volver a ejecutar el análisis y olvidar actualizar una tabla o una cifra en el texto para que el informe deje de ser fiel a los datos.

Los documentos dinámicos resuelven este problema **entrelazando** código y texto en un mismo archivo de texto plano. Al “tejer” (*knit*, en RMarkdown) o “renderizar” (*render*, en Quarto) el documento, el motor ejecuta el código, captura sus resultados (valores, tablas, figuras) y los inserta automáticamente en el lugar correspondiente del texto final. El resultado es un documento cuyo contenido cuantitativo **no puede desincronizarse** del código que lo produjo (Xie et al., 2018).

RMarkdown fue la primera implementación de esta idea en el ecosistema R, construida sobre **knitr** y Pandoc. **Quarto** (Allaire et al., 2022) es su evolución: un sistema de publicación técnica independiente del lenguaje (admite chunks de R, Python, Julia y Observable JS en un mismo documento) desarrollado por Posit. Este mismo libro está escrito en Quarto. Para el trabajo del día a día ambos sistemas comparten la misma filosofía y una sintaxis muy similar; las diferencias principales son:

Tabla 2.1: Comparación entre RMarkdown y Quarto.

Aspecto	RMarkdown (.Rmd)	Quarto (.qmd)
Motor	knitr + Pandoc, invocado desde el paquete R rmarkdown	knitr (o Jupyter) + Pandoc, invocado desde un binario independiente de R
Lenguajes	Principalmente R	R, Python, Julia, Observable JS

Aspecto	RMarkdown (.Rmd)	Quarto (.qmd)
Proyectos multi-documento	Paquete bookdown	Nativo (project: type: book / website)
Extensión de archivo	.Rmd	.qmd

2.2.1. Anatomía de un documento

Tanto un .Rmd como un .qmd se componen de tres bloques:

1. Una **cabecera YAML**, delimitada por líneas `---`, con los metadatos del documento.
2. **Texto** en sintaxis Markdown (encabezados, énfasis, listas, enlaces, imágenes).
3. **Bloques de código** (*chunks*), delimitados por ````{r} ... ```` en RMarkdown (o ````{r}` también en Quarto, aunque Quarto añade opciones de chunk fuera de la cabecera con `#|`).

2.2.1.1. Cabecera YAML

La cabecera define el título, la autoría, la fecha y, sobre todo, el **formato de salida**:

```

1  ---
2  title: "Mi primer documento"
3  author: "Nombre Apellido"
4  date: today
5  output: html_document # RMarkdown
6  # format: html        # Quarto usa 'format' en lugar de 'output'
7  ---

```

2.2.1.2. Texto en Markdown

Markdown es una sintaxis de marcado ligero, diseñada para ser legible incluso sin renderizar. Los elementos más habituales son:

```

1  # Encabezado de nivel 1
2  ## Encabezado de nivel 2
3
4  Texto en **negrita**, en *cursiva*, y `código en línea`.
5
6  - Elemento de lista
7  - Otro elemento
8
9  [Un enlace] (https://quarto.org)
10 ![Una imagen] (ruta/imagen.png)

```

2.2.1.3. Bloques de código

Un *chunk* de código se ejecuta como si fuera una sesión de R independiente, y su salida (impresión de objetos, tablas, gráficos) se inserta en el documento final:

```
“{r Grafico1, eval=FALSE, echo=TRUE, warning=FALSE, message=FALSE, fig.cap='Pie de
figura a gusto'} plot(cars)
```

Las opciones de chunk más utilizadas son:

- **echo**: si se muestra (**TRUE**) u oculta (**FALSE**) el código fuente.
- **eval**: si el chunk se ejecuta (**TRUE**) o solo se muestra como texto (**FALSE**).
- **warning** / **message**: si se muestran los avisos y mensajes que genere el código.
- **fig.cap** (RMarkdown) / **fig-cap** (Quarto): el pie de figura, usado para construir el índice de figuras.

i Por qué el código de este libro no se ejecuta

En el `_quarto.yml` de este libro fijamos `eval: false` a nivel global. Muchos de los ejemplos dependen de datos propios de cada estudiante, de servicios web en directo o de claves de API personales, por lo que ejecutarlos durante la generación del PDF no sería reproducible de forma general. En vuestros propios documentos del seminario, sin embargo, **sí querréis** que el código se ejecute (`eval: true`, el valor por defecto), precisamente para aprovechar la naturaleza dinámica del documento.

2.2.2. Ejemplo mínimo

Cuando creáis un nuevo documento RMarkdown en RStudio (*File* → *New File* → *R Markdown*), la plantilla por defecto incluye un ejemplo mínimo que ilustra el mecanismo, usando el set de datos `cars` incluido en R base:

```
1 summary(cars)

1 plot(pressure)
```

Al “tejer” el documento, ambos chunks se sustituyen por su salida real: la tabla resumen y la figura, respectivamente. Este mecanismo, aplicado a datos biomédicos reales en lugar de a `cars` o `pressure`, es exactamente lo que sustenta el seminario evaluable del curso.

2.3. GitHub como repositorio remoto

El Capítulo 1 trabajó exclusivamente con un repositorio **local**: la carpeta `.git/` vive únicamente en vuestro ordenador. GitHub añade un repositorio **remoto**, alojado en la nube, que permite:

- Tener una copia de seguridad del historial completo fuera de vuestro equipo.
- Compartir el código y su historial con el equipo docente para la evaluación del seminario.

- Colaborar con compañeras y compañeros sobre el mismo proyecto (aunque, en este curso, el seminario se entrega de forma individual o en grupos reducidos).

RStudio incorpora una integración nativa con Git (pestaña *Git* del panel superior derecho) que permite ejecutar `add`, `commit`, `push` y `pull` sin salir del entorno, aunque internamente sigue invocando los mismos comandos vistos en el Capítulo 1. Para la configuración inicial (credenciales SSH o *token* de acceso personal, vinculación de RStudio con una cuenta de GitHub) remitimos a la guía de referencia Bryan (2017), que cubre en detalle cada sistema operativo.

El flujo básico de trabajo con un repositorio remoto añade dos comandos a los ya conocidos:

Listado 2.1 terminal

```
1 git push      # Sube los commits locales al repositorio remoto
2 git pull      # Descarga los commits del remoto que no tenemos en local
```

2.4. Estructura del seminario evaluable

El seminario es el trabajo de evaluación continua del curso: un análisis original de una fuente de datos biomédica, documentado íntegramente en un documento RMarkdown/Quarto y publicado en un repositorio de GitHub. La entrega consta de **tres elementos**:

- I. El **archivo fuente** (`.Rmd` o `.qmd`) con el texto y el código empleado.
- II. El documento **renderizado** (`.html`), que combina texto, código y resultados (tablas, figuras).
- III. La **URL del repositorio de GitHub** donde se ha desarrollado el trabajo.

El documento debe incluir los identificadores básicos (título, autoría, curso) y las siguientes secciones:

2.4.1. Introducción

Contextualiza el seminario: presenta la temática, resume lo que ya se conoce sobre ella y, sobre todo, señala la pregunta concreta que aún no tiene respuesta y que el trabajo pretende abordar.

2.4.2. Objetivo general y objetivos específicos

Un **objetivo general**, expresado en una sola frase, que muestre con claridad la relación entre variables que se va a estudiar. A partir de él, tres o cuatro **objetivos específicos**: normalmente, una o dos preguntas de caracterización de las variables (descripción espacial, temporal, etc.) y una o dos preguntas centradas en la relación o correlación entre ellas. La ausencia de relación o correlación es, en sí misma, un resultado admisible.

2.4.3. Metodología y resultados

Para cada objetivo específico se presenta el código empleado y las representaciones necesarias (tablas, figuras) que permiten responder a ese objetivo. Esta sección constituye, en la práctica, el grueso del documento, y es donde se aplican directamente las técnicas de los Capítulo 5 a Capítulo 7 de este libro.

2.4.4. Conclusiones generales

Sintetiza el conocimiento generado, en relación directa con el objetivo general planteado, reconociendo explícitamente las limitaciones del análisis (tamaño muestral, sesgos de la fuente de datos, ausencia de relaciones causales, etc.).

2.4.5. Referencias

Listado de las fuentes utilizadas para justificar la introducción y para discutir la relevancia de las conclusiones, preferiblemente en formato APA. La gestión de esta bibliografía dentro de RMarkdown/Quarto se trata en detalle en la Sección 9.5.

2.5. Criterios de evaluación

De cara a la evaluación, se atiende especialmente a:

- I. El aspecto general del **repositorio** de GitHub.
- II. La calidad del **README** o página de presentación del repositorio.
- III. La **organización de archivos y carpetas** dentro del repositorio (véanse las buenas prácticas de estructuración de proyectos que se siguen a lo largo de este libro, por ejemplo en el Capítulo 5).
- IV. La lectura del documento **.html** renderizado, descargado y abierto fuera de GitHub, para comprobar que el resultado final —texto, código y salidas— es coherente y está bien resuelto.

3 Objetos en R

3.1. Objetivos

Antes de importar y transformar fuentes de datos biomédicas es imprescindible dominar cómo R representa la información internamente. Este capítulo recorre la jerarquía de tipos y estructuras de datos de R siguiendo el modelo expuesto en Wickham (2019), deteniéndonos en los aspectos que con más frecuencia generan errores sutiles en el análisis de datos: la coerción de tipos, los valores ausentes y la diferencia entre `data.frame` y `tibble`.

3.2. El sistema de objetos de R

Casi todo lo que se manipula en R es, en su forma más básica, un **vector**. R distingue dos grandes familias:

- **Vectores atómicos**: todos sus elementos son del mismo tipo (lógico, entero, doble, carácter, entre otros).
- **Listas** (a veces llamadas *vectores recursivos*): sus elementos pueden ser, a su vez, de cualquier tipo, incluidas otras listas.

Sobre estas dos estructuras base, R construye tipos más especializados añadiendo **atributos**: metadatos adicionales adheridos al vector. Un factor es un vector entero con un atributo `levels`; una fecha es un vector doble con un atributo de clase; un `data.frame` es una lista de vectores de igual longitud con atributos `names`, `row.names` y `class`. Entender esta construcción por capas es la clave para predecir cómo se comportará un objeto en operaciones aritméticas, en comparaciones o al indexarlo.

3.3. Vectores

3.3.1. Vectores atómicos

Todo vector se crea, se imprime y se consulta con un pequeño conjunto de funciones que usaremos constantemente a lo largo del curso:

```
1 x <- 3
2 print(x)
3 x
4 class(x) # la clase "declarada" del objeto
```



```
5 typeof(x) # el tipo interno de almacenamiento
```

`class()` y `typeof()` no siempre coinciden: `class()` describe el comportamiento del objeto de cara al sistema de despacho de funciones de R (S3/S4/R6/S7), mientras que `typeof()` describe cómo se almacena internamente. Para un vector numérico simple ambas coinciden ("`numeric`"), pero para un entero explícito no:

```
1 y <- 3L          # sufijo L: entero explícito
2 class(y)         # "integer"
3 typeof(y)        # "integer"
4
5 z <- 1L + 1L
6 class(z)
7 typeof(z)
```

R dispone de cuatro tipos atómicos principales, que forman una jerarquía de generalidad creciente:

```
1 lgl_var <- c(TRUE, FALSE)          # lógico
2 int_var <- c(1L, 6L, 10L)          # entero
3 dbl_var <- c(1, 2.5, 4.5)          # doble (numérico)
4 chr_var <- c("these are", "some strings") # carácter
5
6 typeof(chr_var)
```

3.3.2. Valores ausentes (NA)

En datos biomédicos, la ausencia de un valor (una determinación analítica no realizada, una pregunta de un cuestionario sin responder) es la norma, no la excepción. R representa la ausencia de información con `NA`, que se propaga por defecto a través de cualquier cálculo:

```
1 x <- c(1, 5, 8, 10)
2 mean(x)
3
4 x <- c(1, 5, NA, 10)
5 mean(x)          # NA: R no asume cómo tratar el hueco
6
7 mean(x, na.rm = TRUE) # se excluyen los NA explícitamente
8
9 is.numeric(x)
10 as.character(x)
```

Este comportamiento —propagar el NA en vez de ignorarlo silenciosamente— es deliberado: obliga a la persona que analiza los datos a decidir explícitamente cómo tratar la información ausente, en lugar de que R lo decida por ella.

3.3.3. Coerción (*coercion*)

Cuando combinamos elementos de distinto tipo en un mismo vector atómico, R los **coacciona** todos al tipo más general presente, siguiendo la jerarquía lógico → entero → doble → carácter:

```

1  c(1, 2, 3)
2  typeof(c(1, 2, 3))
3
4  c(1, 2, "Pablo")
5  typeof(c(1, 2, "Pablo"))    # todo pasa a carácter
6
7  c(TRUE, FALSE, FALSE, 2L, 3.53, "estamos dormidos")
8  typeof(c(TRUE, FALSE, FALSE, 2L, 3.55))    # double
9  typeof(c(TRUE, FALSE, FALSE, 1L, 2, "Tubby"))    # character

```

La coerción también opera de forma silenciosa dentro de funciones matemáticas, lo que resulta muy útil para, por ejemplo, contar cuántos elementos de un vector lógico son TRUE (TRUE se coacciona a 1, FALSE a 0):

```

1  x <- c(FALSE, FALSE, TRUE)
2  is.logical(x)
3  as.numeric(x)
4
5  sum(x)    # número de TRUE
6  mean(x)   # proporción de TRUE

```

Cuando la coerción no es posible (por ejemplo, convertir un texto no numérico a **numeric**), R genera un NA y un aviso, en vez de detener la ejecución:

```

1  as.numeric(c(1, 1.5, "Joaquina"))

```

3.3.4. Nombres

Los elementos de un vector pueden etiquetarse con nombres, almacenados en el atributo **names**:

```

1  x <- c(a = 1, b = 2, c = 3)
2  attributes(x)
3  names(x)
4

```

```

5 x <- c(1:5)
6 names(x) <- c("a", "b", "c", "d", "e")
7
8 x <- setNames(1:3, c("h", "j", "k"))
9
10 x <- unname(x) # elimina los nombres

```

3.3.5. Dimensiones

El atributo `dim` transforma un vector en una estructura con una geometría explícita: sin `dim`, tenemos un vector; con `dim` de longitud 2, una matriz; con `dim` de longitud 3, un *array*. Las tablas de datos (`data.frame`) y las listas heterogéneas cubren, respectivamente, el caso tabular y el caso arbitrariamente anidado:

```

1 x <- c(1:5)
2 dim(x) # NULL: un vector no tiene "dim"
3 length(x)
4
5 m <- matrix(c(1:10, "hola", "pablo"), nrow = 3, ncol = 4)
6 dim(m); class(m); typeof(m)
7
8 df1 <- data.frame(Numeros = 1:5, Letras = letters[1:5])
9 dim(df1); class(df1); typeof(df1)
10
11 l1 <- list(Enteros = 1:50, Letra = "a",
12           Logico = c(TRUE, FALSE, TRUE), Numeros = c(2.3, 5.9))
13 typeof(l1); length(l1); str(l1)

```

3.3.6. Indexación de vectores

R indexa desde 1 (a diferencia de lenguajes como Python o C, que indexan desde 0), y admite indexar por posición, por rango y con funciones auxiliares como `length()`:

```

1 A <- c("a", "b", "c", "d", "e", "f", "g", "h")
2
3 A[3] # tercer elemento
4 A[1:3] # rango: los tres primeros
5 A[length(A)] # último elemento, sin conocer de antemano cuántos hay
6
7 typeof(A[1:3]) # la indexación conserva el tipo del vector original

```

3.3.7. Factores

Un **factor** es la representación en R de una variable categórica: internamente es un vector entero, con un atributo `levels` que traduce cada entero a una etiqueta de texto.

```

1 x <- factor(c("a", "b", "b", "a"))
2 typeof(x)      # "integer" por debajo
3 attributes(x)  # levels y class
4
5 sex_char <- c("m", "m", "m")
6 table(sex_char)
7
8 sex_factor <- factor(sex_char, levels = c("m", "f"),
9                      labels = c("Macho", "Hembra"))
10 table(sex_factor) # a diferencia de table(sex_char), conserva el nivel "Hembra" con con

```

Este último detalle es importante en informes clínicos: un factor con niveles predefinidos muestra explícitamente las categorías sin observaciones (por ejemplo, un grupo de tratamiento sin eventos adversos registrados), mientras que un vector de caracteres simplemente las omite.

3.3.7.1. Factores ordinales

Cuando las categorías tienen un orden natural (estadios de una enfermedad, grados de severidad), usamos `ordered()` en lugar de `factor()`:

```

1 grade <- ordered(c("b", "b", "a", "c"), levels = c("c", "b", "a"))
2 grade

```

3.3.8. Fechas

Los vectores de fecha se construyen sobre vectores dobles: internamente almacenan el número de días transcurridos desde el 1 de enero de 1970 (el *epoch* de Unix).

```

1 today <- Sys.Date()
2 class(today); typeof(today)
3
4 Newdate <- as.Date("1970/02/01")
5 class(Newdate)
6 unclass(Newdate) # el número de días bajo la apariencia de fecha

```

3.3.9. Fechas-horas

R ofrece dos representaciones de fecha-hora: `POSIXct` (un único número, segundos desde el *epoch*; eficiente para almacenar) y `POSIXlt` (una lista con componentes separados —año, mes, día, hora...—; más cómoda para extraer partes concretas). Trabajaremos casi siempre con `POSIXct`:

```
1 now_ct <- as.POSIXct("2018-08-01 22:00", tz = "UTC")
2 typeof(now_ct); attributes(now_ct)
3
4 # El atributo tzone permite reinterpretar el instante en otra zona horaria
5 structure(now_ct, tzone = "Asia/Tokyo")
6 structure(now_ct, tzone = "Australia/Lord_Howe")
```

3.3.10. Duraciones

Una duración representa el intervalo de tiempo entre dos fechas u horas, y se almacena en objetos `difftime`:

```
1 one_week_1 <- as.difftime(1, units = "weeks")
2 one_week_2 <- as.difftime(7, units = "days")
3
4 difftime(now_ct, Sys.Date(), units = "hours")
```

3.4. Matrices y arrays

Una matriz almacena datos homogéneos en dos dimensiones; un *array* generaliza esta idea a dos o más dimensiones. Ambas estructuras siguen siendo, por debajo, vectores atómicos con un atributo `dim`:

```
1 m <- matrix(seq(1:6), nrow = 2, byrow = TRUE)
2 str(m); typeof(m); class(m)
```

3.4.1. Coerción en matrices

Como una matriz sigue siendo un vector atómico, se aplica la misma regla de coerción que en Sección 3.3.3: basta con **un** elemento de carácter para que toda la matriz pase a ser de tipo `character`:

```
1 m1 <- matrix(c(1, 2, 3, 4, 5, "Pablo"), nrow = 2)
2 typeof(m1) # "character"
```

3.4.2. Indexación de matrices

Al existir dos dimensiones, la indexación sigue siempre el orden `[filas, columnas]`:

```

1 m2 <- matrix(seq(1:8), nrow = 4, byrow = TRUE)
2 colnames(m2) <- c("Impares", "Pares")
3
4 m2[2, ]           # segunda fila, todas las columnas
5 m2[, 2]           # todas las filas, segunda columna
6 m2[, "Impares"]   # indexación por nombre de columna
7 m2[c(1, 2, 4), "Pares"]
8
9 # La indexación de una matriz NO cambia su clase...
10 class(m2[1:3, "Pares"])
11 # ...salvo que el resultado pierda una dimensión y quede reducido a un vector
12 typeof(m2[1:3, "Pares"])

```

3.5. Tablas de datos (`data.frame`)

El `data.frame` es, probablemente, la estructura más usada en análisis de datos: es el análogo heterogéneo de la matriz, una tabla en la que cada columna puede tener su propio tipo, siempre que todas tengan la misma longitud.

```

1 df1 <- data.frame(Numeros = 1:5, Letras = letters[1:5])
2 typeof(df1)      # "list": por debajo, un data.frame ES una lista
3 attributes(df1) # names, row.names, class

```

3.5.1. `tibble` frente a `data.frame`

`tibble`, del paquete homónimo dentro del `tidyverse` (Müller & Wickham, 2026), reimplementa el `data.frame` corrigiendo varios comportamientos históricos de R base que generan errores sutiles:

```

1 library(dplyr)
2
3 df2 <- tibble(Numeros = 1:5, Letras = letters[1:5])

```

Las diferencias más relevantes para el trabajo diario son:

- Un `data.frame` **recicla** vectores más cortos para completar el número de filas (`data.frame(x = 1:4, y = "Spain")` repite "Spain" cuatro veces); un `tibble` no recicla vectores de longitud distinta de 1, y avisa si detecta una inconsistencia.

- `df1$Nu` (con un `data.frame`) encuentra la columna `Numeros` por **coincidencia parcial** del nombre; `df2$Nu` (con un `tibble`) devuelve `NULL` y un aviso, evitando que un error tipográfico pase inadvertido:

```
1 str(df1$Nu)    # coincidencia parcial: encuentra "Numeros"
2 str(df2$Nu)    # NULL: tibble exige el nombre completo
```

- Al imprimir un `tibble` en consola, este muestra únicamente las primeras filas y el tipo de cada columna, en lugar de volcar la tabla completa como hace un `data.frame`.

3.5.2. Indexación de `data.frame`

```
1 df1$Numeros      # por nombre de columna
2 df1[1:2, 2]      # por posición [fila, columna]
3 df1[, "Letras"]  # todas las filas, columna por nombre
4 df1[c(1, 3, 5), "Letras"] # filas concretas, columna por nombre
```

3.6. Listas

Las listas son el análogo heterogéneo de los vectores atómicos: cada elemento puede ser de cualquier tipo, incluida otra lista.

```
1 l1 <- list(
2   Enteros = 1:3,
3   Letra = "a",
4   Logico = c(TRUE, FALSE, TRUE),
5   Numeros = c(2.3, 5.9)
6 )
7
8 typeof(l1); str(l1)
```

Un aspecto poco intuitivo, pero importante para trabajar con conjuntos de datos grandes, es que una lista puede contener el **mismo objeto varias veces sin multiplicar el uso de memoria**, gracias a la semántica de copia perezosa (*lazy copy-on-modify*) de R:

```
1 lobstr::obj_size(mtcars)
2
3 l2 <- list(mtcars, mtcars, mtcars, mtcars)
4 lobstr::obj_size(l2)    # mucho menor que 4 veces el tamaño de mtcars
```

3.6.1. Combinación de listas

`c()` combina listas “aplanando” un nivel de anidamiento, a diferencia de `list()`, que preserva la estructura anidada tal cual se le pasa:

```
1  l4 <- list(list(1, 2), c(3, 4))    # dos elementos: una lista y un vector
2  l5 <- c(list(1, 2), c(3, 4))      # cuatro elementos: c() los aplanan
```

3.6.2. Coerción de listas

```
1  is.list(list(1:3))
2  typeof(list(1:3))          # "list"
3
4  as.list(seq(1:3))          # convierte un vector en una lista de un elemento por posición
5  typeof(as.list(seq(1:3)))
```

3.6.3. Indexación de listas

La indexación de listas admite tres notaciones, con un matiz que suele confundir al inicio: `[[ ]]` extrae un **elemento**, mientras que `[ ]` extrae una **sublista** (que sigue siendo una lista, aunque contenga un único elemento):

```
1  l1[[1]]                    # el vector Enteros, "desempaquetado" de la lista
2  l1[[1]][2]                 # el segundo elemento de ese vector
3
4  l1$Logico[3]               # acceso por nombre + indexación del vector resultante
5  l1$Letra
6
7  # Añadir un elemento nuevo a la lista, con o sin nombre explícito
8  l1[[5]] <- df2
9  l1$DataFrame <- df1
```


4 Funciones en R

4.1. Objetivos

R es, ante todo, un lenguaje **funcional**: prácticamente todo lo que hace algo en R —incluidos operadores como `+` o `[]`— es, por debajo, una función (Wickham, 2019). Este capítulo cubre las estructuras de control que gobiernan el flujo de ejecución (condicionales e iteración) y cómo escribir funciones propias, ilustrándolo con ejemplos clínicos y de bioinformática de secuencias.

4.2. Estructuras condicionales

4.2.1. `if`

La estructura `if` ejecuta un bloque de código únicamente cuando una condición lógica se evalúa como `TRUE`. Consideremos la clasificación de la presión arterial sistólica de un paciente (hipertensión definida, en este ejemplo, como una presión sistólica superior a 140 mmHg):

```
1 presion_sistolica <- 145
2
3 if (presion_sistolica > 140) {
4   print("El paciente tiene presión arterial alta.")
5 }
```

4.2.2. `if-else`

Cuando existen varias categorías mutuamente excluyentes, encadenamos `if`, `else if` y `else`. R evalúa las condiciones en orden y ejecuta únicamente la rama de la **primera** condición que se cumple:

```
1 presion_sistolica <- 135
2
3 if (presion_sistolica <= 120) {
4   print("La presión arterial del paciente es normal.")
5 } else if (presion_sistolica <= 140) {
6   print("El paciente tiene presión arterial alta.")
7 } else {
8   print("El paciente tiene presión arterial muy alta.")
9 }
```

```
9 }
```

4.2.3. ifelse(): la versión vectorizada

`if/else` evalúa una **única** condición lógica (de longitud 1). Cuando necesitamos aplicar la misma regla a un vector completo de valores —por ejemplo, clasificar la presión arterial de varios pacientes a la vez— usamos `ifelse()`, que está vectorizada:

```
1 presiones_sistolicas <- c(115, 130, 145, 125, 110)
2
3 clasificacion <- ifelse(
4   presiones_sistolicas <= 120, "Normal",
5   ifelse(presiones_sistolicas <= 140, "Alta", "Muy alta")
6 )
7
8 print(clasificacion)
```

if vs. ifelse()

Un error habitual es usar `if` directamente sobre un vector de longitud mayor que 1: en versiones recientes de R esto genera un error (`the condition has length > 1`). La regla práctica es: **if** para una decisión, **ifelse()** (o, en el estilo **tidyverse**, `dplyr::case_when()`, que veremos en el Capítulo 6) para etiquetar cada elemento de un vector.

4.3. Iteración

4.3.1. La familia `apply`

R base ofrece una familia de funciones que aplican una función a subconjuntos o márgenes de una estructura de datos, evitando escribir el bucle explícitamente. Son el precedente directo de funciones como `dplyr::across()` o `purrr::map()` del **tidyverse**.

4.3.1.1. `tapply()`: agregación por grupos

`tapply()` aplica una función a un vector, agrupado según los niveles de otra variable. Es el equivalente en R base de `group_by()` |> `summarise()` (Capítulo 6):

```
1 datos <- data.frame(
2   paciente = 1:10,
3   glucosa = c(120, 135, 140, 125, 130, 155, 145, 160, 150, 165),
4   imc_categoria = c("Normal", "Sobrepeso", "Obesidad", "Normal", "Sobrepeso",
5                     "Obesidad", "Obesidad", "Sobrepeso", "Normal", "Obesidad")
6 )
```

```

6 )
7
8 media_glucosa <- tapply(datos$glucosa, datos$imc_categoria, mean)
9 print(media_glucosa)

```

4.3.1.2. apply(): aplicar por filas o columnas

`apply()` aplica una función a lo largo de una dimensión (`MARGIN = 1` para filas, `MARGIN = 2` para columnas) de una matriz o *array*:

```

1 glucosa_matriz <- matrix(
2   c(120, 135, 140, 125, 130, 155, 145, 160, 150, 165, 170, 175),
3   nrow = 4, byrow = TRUE,
4   dimnames = list(paciente = paste("Paciente", 1:4),
5                     momento  = paste("Momento", 1:3))
6 )
7
8 media_por_paciente <- apply(glucosa_matriz, MARGIN = 1, FUN = mean)
9 print(media_por_paciente)

```

4.3.2. while: iteración condicionada

Un bucle **while** repite un bloque de código **mientras** una condición se mantenga verdadera. Es apropiado cuando no sabemos de antemano cuántas iteraciones harán falta, como en la simulación de un seguimiento clínico hasta alcanzar un valor objetivo:

```

1 nivel_objetivo <- 120
2 nivel_glucosa <- 150
3 contador <- 0
4
5 while (nivel_glucosa > nivel_objetivo) {
6   contador <- contador + 1
7   nivel_glucosa <- nivel_glucosa - sample(1:10, 1)
8   cat("Lectura", contador, ": Nivel de glucosa =", nivel_glucosa, "mg/dL\n")
9 }
10
11 cat("Objetivo alcanzado tras", contador, "lecturas.\n")

```

4.3.3. for: iteración sobre una secuencia conocida

4.3.3.1. for simple

Cuando el número de iteraciones es conocido de antemano —por ejemplo, recorrer una lista fija de pacientes—, `for` es la estructura natural:

```

1  num_habitaciones <- 5
2  pacientes <- c("Paciente1", "Paciente2", "Paciente3",
3                "Paciente4", "Paciente5", "Paciente6")
4
5  asignacion <- vector("list", length(pacientes))
6
7  for (i in seq_along(pacientes)) {
8    habitacion <- ((i - 1) %% num_habitaciones) + 1
9    asignacion[[i]] <- paste(pacientes[i], "se asigna a la habitación", habitacion)
10 }
11
12 for (asignacion_paciente in asignacion) {
13   cat(asignacion_paciente, "\n")
14 }
```

i `seq_along(x)` en lugar de `1:length(x)`

La guía de estilo `tidyverse` (Wickham, 2020) recomienda `seq_along(x)` frente a `1:length(x)` para generar el índice de un bucle: si `x` tiene longitud 0, `1:length(x)` genera erróneamente la secuencia `c(1, 0)`, mientras que `seq_along(x)` genera correctamente una secuencia vacía, evitando iteraciones fantasma difíciles de depurar.

4.3.3.2. for anidado

Un bucle `for` puede anidarse dentro de otro para recorrer combinaciones de dos (o más) dimensiones, como asignar varias fechas de examen a cada paciente:

```

1  pacientes <- c("Paciente1", "Paciente2", "Paciente3")
2  fechas_examen <- c("2024-09-01", "2024-09-15", "2024-10-01")
3
4  programacion_examenes <- list()
5
6  for (paciente in pacientes) {
7    programacion_examenes[[paciente]] <- vector("list", length(fechas_examen))
8    for (fecha in fechas_examen) {
9      programacion_examenes[[paciente]][[fecha]] <-
10       paste("Examen de radiografía programado para", fecha)
```

```

11   }
12 }
13
14 for (paciente in names(programacion_exámenes)) {
15   cat("\n", paciente, ":\n")
16   for (examen in programacion_exámenes[[paciente]]) {
17     cat(" -", examen, "\n")
18   }
19 }

```

! Bucles explícitos frente a vectorización

R está optimizado internamente para operaciones **vectorizadas**: la mayoría de sus funciones matemáticas y lógicas ya operan elemento a elemento sin necesidad de un `for` explícito (`presiones_sistolicas * 2`, por ejemplo, no requiere ningún bucle). Los bucles `for/while` son imprescindibles cuando cada iteración depende del resultado de la anterior (como en el ejemplo de `while` anterior) o cuando el efecto que buscamos no es devolver un valor sino producir un efecto secundario (escribir en disco, imprimir en pantalla). Para el resto de casos, preferid siempre las funciones vectorizadas o la familia `apply/across()`: son más legibles y, habitualmente, más rápidas.

4.4. Funciones propias

4.4.1. Anatomía de una función en R

Una función en R se define con `function()` y se asigna a un nombre como cualquier otro objeto. Tiene tres componentes: sus **argumentos** (`formals()`), su **cuerpo** (`body()`) y el **entorno** en el que fue creada (`environment()`), que determina qué variables puede “ver” además de las que recibe como argumento (Wickham, 2019). La guía de estilo `tidyverse` recomienda nombrar las funciones con verbos en minúscula y `snake_case` (por ejemplo, `contar_nucleotidos()` en lugar de `NucleotideCount` o `contarNucleotidos`) (Wickham, 2020).

4.4.2. Ejemplo: análisis de secuencias genéticas

Ilustramos la definición de funciones propias con un caso de bioinformática elemental: el trabajo con una secuencia de ADN representada como una cadena de texto.

```

1  seq_dna <- c("ATGTCACCACAAACAGAGACT")

```

4.4.2.1. Conteo de nucleótidos

```

1  contar_nucleotidos <- function(secuencia_dna = seq_dna) {
2    conteo_nucleotidos <- unlist(strsplit(x = secuencia_dna, split = "", fixed = TRUE))
3    print(table(conteo_nucleotidos))
4  }
5
6  contar_nucleotidos(secuencia_dna = seq_dna)

```

Nótese el uso de un **valor por defecto** (`secuencia_dna = seq_dna`) en la definición: si la función se llama sin argumentos, opera sobre `seq_dna`, pero admite cualquier otra secuencia sin modificar el código de la función.

4.4.2.2. Cadena complementaria

En una doble hélice de ADN, cada base se empareja con su complementaria: adenina (A) con timina (T), y citosina (C) con guanina (G). `chartr()` sustituye caracteres de acuerdo con esa correspondencia:

```

1  complementaria <- function(secuencia_dna = seq_dna) {
2    complementaria_adn <- chartr("ATGC", "TACG", secuencia_dna)
3    complementaria_adn <- paste(complementaria_adn, collapse = "")
4    return(complementaria_adn)
5  }
6
7  complementaria(seq_dna)

```

4.4.2.3. Transcripción

La transcripción convierte una secuencia de ADN en su ARN mensajero correspondiente, en el que la timina (T) es reemplazada por uracilo (U):

```

1  transcripcion <- function(secuencia_dna = seq_dna) {
2    seq_arn <- chartr("T", "U", secuencia_dna)
3    seq_arn <- paste(seq_arn, collapse = "")
4    return(seq_arn)
5  }
6
7  transcripcion(seq_dna)

```

4.4.3. Modularizar código: `source()`

A medida que un proyecto crece, conviene separar las funciones propias del script principal de análisis, guardándolas en archivos `.R` independientes (por ejemplo, `INPUT/FUNCTIONS/Transcripcion.R`) e importándolas con `source()`:

```
1 source("INPUT/FUNCTIONS/Transcripcion.R")
2 transcripcion(seq_dna)
```

Esta separación entre “funciones” (código reutilizable, sin efectos secundarios) y “análisis” (código específico de un proyecto, que sí produce salidas) es una de las prácticas de organización de proyectos más recomendadas para mantener el código de un seminario ordenado y fácil de revisar (Capítulo 2).

5 Importación y exportación de datos

5.1. Objetivos

Ninguna fuente de datos biomédica sirve de nada si no sabemos leerla. Este capítulo recorre la importación de datos **estructurados** (hojas de cálculo, CSV) y **semiestructurados** (JSON, XML), y su exportación de vuelta a estos formatos. Dedicamos especial atención al formato **JSON**, por ser el formato de intercambio dominante en las API web que iremos usando a lo largo del curso (PubMed en el Capítulo 9, *endpoints* SPARQL en el Capítulo 8, y buena parte de los portales de datos abiertos en salud y medio ambiente).

Paquetes usados en este capítulo: `readxl` (Wickham & Bryan, 2026), `readr` (Wickham, Hester, & Bryan, 2026), `jsonlite` (Ooms, 2025), `tidyjson` (Stanley, 2026), `rjson` (Couture-Beil, 2024), `XML` (CRAN Team, 2026) y `xml2` (Wickham, Hester, & Ooms, 2026).

5.2. Datos estructurados: hojas de cálculo y CSV

5.2.1. Importar desde Excel

`readxl::read_excel()` lee archivos `.xls/.xlsx` sin necesidad de tener Excel instalado (a diferencia de paquetes más antiguos, que dependían de Java o de la propia aplicación). Podemos indicar la hoja concreta a leer con el argumento `sheet`:

```
1 library(readxl)
2
3 Galapagos <- read_excel("INPUT/DATA/Galapagos_DB.xlsx", sheet = "DBR")
4
5 str(Galapagos)
6 summary(Galapagos)
```

5.2.2. Importar desde CSV

`readr::read_delim()` (y sus variantes especializadas `read_csv()`, `read_csv2()`) leen archivos de texto delimitado, adivinando automáticamente el tipo de cada columna a partir de una muestra de los datos:


```

1 library(readr)
2
3 Acc_Car <- read_delim("INPUT/DATA/accidentalidad-por-carreteras.csv",
4                       delim = ";", escape_double = FALSE, trim_ws = TRUE)
5
6 summary(Acc_Car)
7 table(Acc_Car$T.RED)

```

5.2.2.1. Control explícito de tipos de columna

Dejar que `readr` adivine el tipo de cada columna es cómodo, pero puede llevar a que una variable categórica (por ejemplo, el tipo de red viaria) se importe como texto libre en lugar de como factor. El argumento `col_types`, junto con las funciones `col_*()`, permite fijar el tipo explícitamente:

```

1 Acc_Car <- read_delim(
2   "INPUT/DATA/accidentalidad-por-carreteras.csv",
3   delim = ";", escape_double = FALSE, trim_ws = TRUE,
4   col_types = cols(T.RED = readr::col_factor(levels = NULL))
5 )

```

Buena práctica

Fijar `col_types` de forma explícita no es solo una cuestión de estilo: evita que una columna cambie silenciosamente de tipo si una futura actualización del archivo fuente añade, por ejemplo, un valor no numérico en una columna que antes solo contenía números.

5.2.3. Importar datos directamente desde la web

`read_delim()` (como la mayoría de funciones de importación de `readr`) acepta una URL exactamente igual que una ruta local, siempre que el recurso remoto sea directamente descargable:

```

1 Agua_Consumo <- read_delim(
2   "https://datosabiertos.jcyl.es/web/jcyl/risp/es/salud/calidad-aguas-consumo/12848397890
3   delim = ";"
4 )
5
6 Agua_Consumo$`Nº infraestructuras - Depósitos`

```

Nótese el uso de comillas invertidas (```) para referirnos a columnas cuyo nombre contiene espacios o caracteres especiales: es sintaxis estándar de R (no exclusiva del `tidyverse`), necesaria en cuanto el nombre de una variable deja de ser un identificador válido por sí mismo.

5.3. Datos semiestructurados: por qué importan en biomedicina

Los datos **estructurados** (una tabla CSV, una hoja de Excel) tienen un esquema fijo: todas las filas comparten exactamente las mismas columnas. Muchas fuentes de datos biomédicas y de la Web Semántica, en cambio, producen datos **semiestructurados**: tienen una organización jerárquica reconocible (objetos con propiedades, listas de elementos), pero esa jerarquía puede variar de un registro a otro, anidarse a varios niveles, o incluir campos opcionales. Es el caso de:

- Las respuestas de prácticamente cualquier **API REST** moderna (PubMed/NCBI E-utilities, DBpedia, Wikidata, portales de datos abiertos), que se sirven en **JSON**.
- Los documentos **RDF/JSON-LD**, que veremos en el Capítulo 8, donde el propio JSON codifica un grafo de tripletas.
- Los registros de historia clínica electrónica en estándares como **HL7 FHIR**, que se representan tanto en XML como en JSON.

Forzar estos datos a una tabla rectangular *antes* de leerlos no es una opción: primero hay que leerlos respetando su estructura jerárquica, y decidir *después*, con criterio, qué parte de esa jerarquía convertir en columnas de una tabla. Esa es, precisamente, la tarea de `jsonlite` y `tidyjson`.

5.4. JSON: JavaScript Object Notation

5.4.1. Sintaxis y modelo de datos

JSON (*JavaScript Object Notation*) es un formato de texto para el intercambio de datos, estandarizado tanto por el IETF (RFC 8259, Bray (2017)) como por Ecma International (ECMA-404, Ecma International (2017)). A pesar de su nombre, es independiente de JavaScript: es simplemente una sintaxis de texto, legible por personas, que cualquier lenguaje de programación puede generar y analizar.

El formato define exactamente **seis** tipos de valor, que se combinan recursivamente:

Tabla 5.1: Los seis tipos de valor definidos por la especificación JSON (Bray, 2017).

Tipo JSON	Sintaxis	Ejemplo
Objeto	<code>{ "clave": valor, ... }</code>	<code>{"nombre": "Ana", "edad": 34}</code>
<i>Array</i>	<code>[valor, valor, ...]</code>	<code>[1, 2, 3]</code>
Cadena (<i>string</i>)	Texto entre comillas dobles	<code>"Ana"</code>
Número	Entero o decimal, notación científica opcional	<code>34, -2.5, 1.2e3</code>
Booleano	<code>true</code> / <code>false</code> (en minúsculas)	<code>true</code>
Nulo	<code>null</code>	<code>null</code>

Un ejemplo de documento JSON que describe a un paciente, combinando estos seis tipos:

```

1  {
2    "id_paciente": "P-00234",
3    "edad": 47,
4    "fumador": false,
5    "diagnosticos": ["hipertensión", "diabetes tipo 2"],
6    "ultima_revision": null,
7    "constantes": {
8      "presion_sistolica": 132,
9      "presion_diastolica": 85,
10     "frecuencia_cardiaca": 76
11   }
12 }
```

Este ejemplo mínimo ya contiene los tres rasgos que distinguen a JSON de una tabla CSV: **anidamiento** (`constantes` es un objeto dentro del objeto principal), **campos de tipo lista** (`diagnosticos` es un *array* de longitud variable) y **valores nulos explícitos** (`ultima_revision`). Ninguno de los tres tiene una representación directa y sin ambigüedad en una tabla rectangular.

5.4.2. JSON frente a las estructuras de datos de R

La correspondencia natural entre JSON y R, que siguen tanto `jsonlite` como `rjson`, es la siguiente:

Tabla 5.2: Correspondencia entre los tipos JSON y las estructuras de R usada por `jsonlite` (Ooms, 2014).

JSON	Estructura de R
Objeto { }	Lista con nombres (<code>list(clave = valor, ...)</code>)
<i>Array</i> de valores del mismo tipo escalar	Vector atómico
<i>Array</i> de objetos con las mismas claves	<code>data.frame</code> (una fila por objeto)
<i>Array</i> heterogéneo o de longitud irregular	Lista sin nombres
Cadena	<code>character</code>
Número	<code>double</code>
Booleano	<code>logical</code>
<code>null</code>	NULL (o NA dentro de un vector)

La fila más delicada de la Tabla 5.2 es la de los *arrays* de objetos: `jsonlite` los convierte en un `data.frame` **solo si puede**, es decir, solo si todos los objetos del *array* comparten exactamente las mismas claves. En cuanto un registro tiene un campo de más, de menos, o un *array* anidado en su interior, `jsonlite` no puede aplanarlo automáticamente y lo deja como lista. Este es el motivo por el que existe `tidyjson`, que veremos más adelante.

5.4.3. El paquete jsonlite

`jsonlite` (Ooms, 2014, 2025) es, junto con `rjson` (Couture-Beil, 2024), uno de los analizadores de JSON más usados en R, y el que mejor se integra con el `tidyverse` y con `data.frame/tibble`. Su función principal para lectura, `fromJSON()`, acepta indistintamente una cadena de texto, una ruta de archivo local o una URL:

```
1 library(jsonlite)
2
3 # Desde texto
4 fromJSON('{"nombre": "Ana", "edad": 34}')
```

5

```
6 # Desde archivo local
7 Acc_Car_Json <- fromJSON("INPUT/DATA/accidentalidad-por-carreteras.json")
8
9 # Desde una URL (jsonlite descarga y analiza en un solo paso)
10 # fromJSON("https://api.ejemplo.org/pacientes")
```

5.4.3.1. Simplificación automática

Por defecto, `fromJSON()` intenta **simplificar** la estructura resultante lo máximo posible: convierte objetos en vectores con nombre cuando puede, y *arrays* de objetos homogéneos directamente en `data.frame`. Este comportamiento se controla con los argumentos `simplifyVector` y `simplifyDataFrame`:

```
1 # Comportamiento por defecto: simplifica a data.frame si es posible
2 fromJSON("INPUT/DATA/accidentalidad-por-carreteras.json")
3
4 # Sin simplificar: conserva la estructura de lista anidada tal cual
5 fromJSON("INPUT/DATA/accidentalidad-por-carreteras.json", simplifyDataFrame = FALSE)
```

Trabajar con `simplifyDataFrame = FALSE` es útil para *inspeccionar* la estructura real del JSON antes de decidir cómo aplanarlo; para el trabajo habitual, la simplificación por defecto suele ser justo lo que necesitamos.

5.4.3.2. Escritura de JSON: toJSON()

La operación inversa, convertir un objeto de R a texto JSON, se realiza con `toJSON()`:

```
1 x <- list(
2   alpha = 1:5,
3   beta  = "Bravo",
4   gamma = list(a = 1:3, b = NULL),
5   delta = c(TRUE, FALSE)
```

```

6 )
7
8 JSON_x <- toJSON(x, pretty = TRUE)
9 cat(JSON_x)

1 {
2   "alpha": [1, 2, 3, 4, 5],
3   "beta": ["Bravo"],
4   "gamma": {
5     "a": [1, 2, 3],
6     "b": {}
7   },
8   "delta": [true, false]
9 }

```

Dos argumentos de `toJSON()` merecen mención especial:

- `pretty = TRUE` indenta el JSON resultante para que sea legible por humanos (sin él, `toJSON()` produce una única línea compacta, más adecuada para transmitir por red).
- `auto_unbox = TRUE` convierte los vectores de longitud 1 en escalares JSON en lugar de *arrays* de un elemento ("Bravo" en lugar de ["Bravo"]), lo que suele ser el resultado esperado al serializar un único registro:

```

1 toJSON(list(nombre = "Ana", edad = 34), auto_unbox = TRUE, pretty = TRUE)

```

Podemos comprobar que la conversión de ida y vuelta (*round-trip*) es coherente, y guardar el resultado en un archivo `.json`:

```

1 fromJSON(JSON_x) # recupera el objeto de R original
2
3 write(x = JSON_x, file = "OUTPUT/DATA/JSON_x.json")
4
5 # También podemos serializar un data.frame completo
6 jsonstarwars <- toJSON(dplyr::starwars, pretty = TRUE)
7 write(x = jsonstarwars, file = "OUTPUT/DATA/StarWars.json")

```

i jsonlite frente a rjson

El paquete `rjson` (Couture-Beil, 2024) es más antiguo y minimalista: su `fromJSON()/toJSON()` no simplifican automáticamente a `data.frame` y no tienen soporte nativo para leer directamente desde una URL. Ambos paquetes exportan una función `fromJSON()` y una función `toJSON()` con el **mismo nombre**, por lo que si cargáis los dos paquetes a la vez (`library(rjson)` después de `library(jsonlite)`), el último paquete cargado “enmascara” al anterior. Si necesitáis usar ambos en el mismo script, cualificad

la llamada explícitamente: `jsonlite::fromJSON()` frente a `rjson::fromJSON()`. En este libro usaremos `jsonlite` como paquete de referencia, por su mejor integración con el resto del `tidyverse` y por ser el que emplean la mayoría de las fuentes recomendadas para el curso.

5.4.4. JSON anidado: el paquete `tidyjson`

`fromJSON()` resuelve bien el caso en el que el JSON de partida ya es, esencialmente, una tabla disfrazada. Pero en cuanto los registros son heterogéneos —algunos objetos tienen un campo que otros no tienen, o un campo es a veces un escalar y a veces un *array*—, la simplificación automática de `jsonlite` deja de funcionar y el resultado es una lista profundamente anidada, difícil de explorar.

`tidyjson` (Stanley, 2026) resuelve este problema aplicando la filosofía *tidy* al propio proceso de aplanado: en lugar de intentar adivinar la tabla final de un solo paso, representa el JSON como un `tibble` con **una fila por documento JSON** y una columna `document.id`, y ofrece verbos que permiten “descender” nivel a nivel por la jerarquía, de forma explícita y auditable en cada paso —igual que `dplyr` permite encadenar transformaciones tabulares paso a paso (Capítulo 6).

```
1 library(tidyjson)
2
3 data("worldbank") # conjunto de datos de ejemplo incluido en tidyjson
```

5.4.4.1. `spread_all()`: aplanar los campos escalares

`spread_all()` extrae automáticamente todos los campos que son escalares (cadena, número, booleano) en columnas nuevas, ignorando (de momento) los que sean *arrays* u objetos anidados:

```
1 worldbank %>%
2   spread_all()
```

5.4.4.2. Detectar qué queda sin aplanar

Antes de dar por bueno un `spread_all()`, conviene comprobar si queda algún campo de tipo `array` u `object` sin resolver. `gather_object()` convierte cada clave en una fila, y `json_types()` etiqueta el tipo JSON de cada valor:

```
1 worldbank %>%
2   spread_all() %>%
3   gather_object() %>%
4   json_types() %>%
5   count(name, type)
```

En el caso de `worldbank`, esta comprobación revela que el campo `majorsector_percent` sigue siendo de tipo `array`: es una lista de sectores de inversión (con su nombre y su porcentaje) por cada proyecto, y por tanto de longitud variable entre proyectos. `spread_all()` no puede —ni debe— aplanarlo automáticamente, porque no hay una única fila razonable a la que asignar varios sectores.

5.4.4.3. `enter_object()` + `gather_array()`: descender en un *array* anidado

Para aplanar ese campo, “entramos” explícitamente en el objeto correspondiente y desplegamos el *array* en filas, una por elemento:

```
1 worldbank %>%
2   enter_object(majorsector_percent) %>%
3   gather_array() %>%
4   spread_all() %>%
5   select(-document.id, -array.index)
```

Encadenando estas operaciones, podemos calcular, por ejemplo, la inversión media por región y sector, recorriendo explícitamente el pipeline paso a paso:

```
1 worldbank %>%
2   spread_all() %>%
3   select(region = regionname, funding = totalamt) %>%
4   enter_object(majorsector_percent) %>%
5   gather_array() %>%
6   spread_all() %>%
7   rename(sector = Name, percent = Percent) %>%
8   group_by(region, sector) %>%
9   summarize(funding = mean(percent, na.rm = TRUE))
```

5.4.4.4. Ejemplo aplicado: accidentalidad por carreteras (JCyL)

Apliquemos el mismo procedimiento al archivo JSON de accidentalidad por carreteras que ya leímos con `jsonlite`, y comparemos el resultado con la versión ya tabular obtenida a partir del CSV equivalente:

```
1 Acc_Car_Json <- fromJSON("INPUT/DATA/accidentalidad-por-carreteras.json", simplifyVector = FALSE)
2
3 # Convertimos a tibble "tidyjson" y comprobamos que no queden arrays u objetos
4 Acc_Car_Json %>%
5   gather_object() %>%
6   json_types() %>%
7   count(name, type)
8
```

```

9 # Al no haber campos de tipo array/object, spread_all() ya es la tabla final
10 Acc_Car_TJson <- Acc_Car_Json %>% spread_all()
11
12 lobstr::obj_size(Acc_Car_TJson)
13 lobstr::obj_size(Acc_Car) # la versión leída directamente del CSV equivalente

```

💡 tidyjson frente a jsonlite: ¿cuándo usar cada uno?

Como regla práctica: si el JSON de origen ya representa una tabla razonablemente homogénea (todos los registros comparten los mismos campos escalares), `jsonlite::fromJSON()` es más rápido de escribir y suele bastar por sí solo. En cuanto el JSON contiene *arrays* de longitud variable, objetos anidados a varios niveles, o registros con campos opcionales —algo habitual en respuestas de API biomédicas—, `tidyjson` ofrece un camino más controlado, verbo a verbo, que evita sorpresas y hace explícita cada decisión de aplanado.

5.4.5. Buenas prácticas al trabajar con JSON

- **Inspeccionad antes de aplanar.** `str()` sobre el resultado de `fromJSON(..., simplifyVector = FALSE)`, o el patrón `gather_object() |> json_types() |> count(name, type)` de `tidyjson`, os dirán qué campos son escalares y cuáles son *arrays* u objetos, antes de decidir cómo tratarlos.
- **No deis por hecho un esquema fijo.** A diferencia de un CSV, dos documentos JSON de la misma API pueden tener campos distintos (por ejemplo, un campo opcional que solo aparece si el paciente tiene alguna alergia registrada). Comprobad varios registros de ejemplo, no solo el primero.
- **Cread la URI/URL base explícitamente** cuando el JSON alimenta después un flujo de trabajo RDF (Capítulo 8): un identificador numérico interno (`"id": 234`) no es una URI global; conviene transformarlo pronto (`paste0("https://miportal.org/paciente/", id)`).
- **Guardad una copia local** de las respuestas de API antes de procesarlas (`write(JSON_x, "respuesta.json")`), especialmente si el servicio remoto tiene límites de peticiones o requiere una clave de API: así podéis volver a analizar los datos sin repetir la consulta.

5.5. XML

XML (*eXtensible Markup Language*) es el otro gran formato semiestructurado, anterior a JSON y aún ampliamente usado en biomedicina (por ejemplo, en las respuestas del *E-utilities* de PubMed que usaremos en el Capítulo 9, o en el estándar clínico HL7 CDA). A diferencia de JSON, XML distingue entre elementos y atributos, y no tiene un tipo de dato “array” nativo: la repetición de una etiqueta es lo que codifica una lista de elementos.


```

1 library(xml2)
2
3 file_url <- "https://www.w3schools.com/xml/simple.xml"
4 Data <- read_xml(file_url)
5
6 str(Data)    # poca información legible: no es una estructura tabular

```

El paquete XML (CRAN Team, 2026) ofrece `xmlParse()` y `xmlToDataFrame()` para convertir directamente un documento XML simple (sin atributos anidados relevantes) en un `data.frame`:

```

1 library(XML)
2
3 Data_mxl <- xmlParse(Data)
4 DF_xml <- xmlToDataFrame(doc = Data_mxl, stringsAsFactors = FALSE)
5
6 DF_xml %>%
7   as_tibble() %>%
8   transmute(
9     name = factor(name),
10    price_dollars = parse_number(price),
11    description = description,
12    calories = as.numeric(calories)
13  )

```

Igual que en JSON, en cuanto el XML tiene una estructura verdaderamente anidada (atributos, elementos repetidos con distinto significado según el contexto), `xmlToDataFrame()` deja de ser suficiente y hace falta recorrer el árbol XML con `xml2::xml_find_all()/xml_attr()`, de forma análoga a como `tidyjson` recorre un JSON anidado.

5.6. Exportación de datos

5.6.1. Guardar objetos de R: formato nativo (.rds)

Cuando el destino de los datos es **otra sesión de R** (no un sistema externo), el formato más eficiente es el nativo de R, `.rds`, que preserva exactamente la clase y los atributos del objeto (factores, fechas, tipos de columna) sin necesidad de volver a especificarlos al leerlo:

```

1 Data_WorldBank <- worldbank %>% spread_all()
2
3 saveRDS(object = Data_WorldBank, file = "OUTPUT/DATA/WorldBankData.rds")
4
5 rm(Data_WorldBank)
6 Data_WorldBank <- readRDS(file = "OUTPUT/DATA/WorldBankData.rds")

```

5.6.2. Exportar a texto plano (CSV)

Para compartir datos con otros programas (Excel, Python, bases de datos), `readr::write_csv()/write_delim()` son la vía habitual:

```
1 write_csv(x = Data_WorldBank, file = "OUTPUT/DATA/WorldBankData.csv")
2 write_delim(x = Data_WorldBank, file = "OUTPUT/DATA/WorldBankData.txt", delim = ",")
```

5.6.3. Exportar a JSON

`jsonlite::write_json()` (o `toJSON()` seguido de `write()`, como vimos antes) permite el camino inverso al de `fromJSON()`: convertir un `data.frame/tibble` de R en un documento JSON, por ejemplo para alimentar una API propia o publicar un conjunto de datos abierto:

```
1 write_json(Data_WorldBank, "OUTPUT/DATA/WorldBankData.json", pretty = TRUE)
```

5.6.4. Exportar a XML

```
1 write_xml(x = Data, file = "OUTPUT/DATA/Data.xml")
```

6 Procesamiento y manejo de datos

6.1. Objetivos

Una vez importados (Capítulo 5), los datos rara vez están en la forma exacta que necesitamos para responder una pregunta concreta. Este capítulo cubre `dplyr` (Wickham, François, et al., 2026) y `tidyr` (Wickham et al., 2025), los dos paquetes del `tidyverse` dedicados a transformar tablas: filtrar, resumir, reestructurar y combinar. Ambos comparten una misma filosofía de diseño: cada función es un **verbo** que hace una única cosa bien, y los verbos se encadenan mediante el operador *pipe* para construir transformaciones complejas a partir de pasos simples y legibles (Wickham et al., 2019).

Trabajaremos principalmente con `starwars`, el conjunto de datos de personajes de *Star Wars* incluido en `dplyr`, por su tamaño reducido y su mezcla de variables numéricas, categóricas y de tipo lista, ideal para ilustrar todos los verbos sin necesitar datos externos.

```
1 library(dplyr)
2 data("starwars")
```

6.2. Verbos de una tabla: la dimensión de las filas

6.2.1. `filter()`: seleccionar filas por condición

`filter()` conserva las filas para las que una condición lógica es TRUE:

```
1 filter(starwars, mass > 100)           # personajes con mass > 100 kg
2 filter(starwars, mass < 50)
3 filter(starwars, between(mass, 50, 100)) # equivalente legible a mass >= 50 & mass <= 100
4 filter(starwars, skin_color == "blue, grey")
```

6.2.2. `slice()` y su familia: seleccionar filas por posición

Mientras que `filter()` selecciona filas según una condición sobre sus valores, la familia `slice_*`() las selecciona según su **posición** u **orden**:

```
1 slice_head(starwars, n = 5)           # primeras 5 filas
2 slice_tail(starwars, n = 5)           # últimas 5 filas
```

```

3 slice(starwars, 8:12)           # filas 8 a 12
4 slice_sample(starwars, prop = 0.7) # muestra aleatoria del 70% de las filas
5 slice_max(starwars, order_by = height, n = 3) # 3 personajes más altos

```

6.2.3. arrange(): ordenar filas

```

1 arrange(starwars, height)           # ascendente (por defecto)
2 arrange(starwars, desc(height))     # descendente
3 arrange(starwars, desc(height), mass) # por más de una variable, en orden de prioridad

```

6.2.4. summarise(): reducir una tabla a un resumen

`summarise()` colapsa todas las filas de la tabla en una única fila, aplicando una función de agregación (`mean()`, `sd()`, `n()`, etc.) a las columnas indicadas:

```

1 summarise(starwars,
2           Alt_prom = mean(height, na.rm = TRUE),
3           Alt_desv = sd(height, na.rm = TRUE))

```

6.3. Verbos de una tabla: la dimensión de las columnas

6.3.1. select(): elegir columnas

```

1 select(starwars, hair_color, skin_color, eye_color) # por nombre
2 select(starwars, hair_color:birth_year)             # por rango
3 select(starwars, !(hair_color:birth_year))          # negando un rango
4 select(starwars, ends_with("color"))                # por patrón del nombre

```

6.3.2. rename(): renombrar columnas

```

1 rename(starwars, altura = height, peso = mass, color_pelo = hair_color)
2 rename_with(starwars, toupper, ends_with("color")) # renombrado condicionado a un patrón

```

6.3.3. mutate(): crear (o modificar) columnas

`mutate()` añade columnas nuevas conservando el resto de la tabla; `transmute()` hace lo mismo pero descarta las columnas no mencionadas explícitamente:

```

1 mutate(starwars, Altura_m = height / 100)      # altura en metros en vez de centímetros
2 transmute(starwars, Altura_m = height / 100)    # igual, pero solo conserva Altura_m

```

6.3.4. relocate(): reordenar columnas

```

1 relocate(starwars, sex:homeworld, .before = height)

```

6.3.5. case_when(): recodificación condicional con múltiples criterios

`case_when()` generaliza `ifelse()` (Capítulo 4) a un número arbitrario de condiciones, evaluadas en orden hasta encontrar la primera que se cumple; `.default` fija el valor para las filas que no cumplen ninguna:

```

1 starwars %>%
2   select(name:mass, gender, species) %>%
3   mutate(type = case_when(
4     height > 200 | mass > 200 ~ "large",
5     species == "Droid"        ~ "robot",
6     .default = "other"
7   ))

```

6.4. Encadenar transformaciones: el operador *pipe*

El operador *pipe* (`%>%`, del paquete `magrittr`, o su equivalente nativo de R `|>`) toma el resultado de la expresión a su izquierda y lo pasa como primer argumento a la función de su derecha. Encadenando varios verbos con *pipes* obtenemos transformaciones complejas que se leen de arriba abajo, en el mismo orden en que ocurren conceptualmente:

```

1 starwars %>%
2   select(name, species) %>%
3   filter(mass > 77)

```

6.4.1. Agrupación: group_by()

`group_by()` no cambia los datos visibles, pero marca la tabla para que las operaciones posteriores (sobre todo `summarise()` y `mutate()`) se apliquen **por separado a cada grupo**:

```

1 starwars %>%
2   group_by(species, sex) %>%
3   summarise(

```

```

4     height = mean(height, na.rm = TRUE),
5     mass   = mean(mass,   na.rm = TRUE)
6   )

```

Un ejemplo con relevancia clínica directa: el cálculo del índice de masa corporal (IMC) y la identificación de los tres personajes con el IMC más alto:

```

1 starwars %>%
2   mutate(
3     height_m = height / 100,
4     BMI = mass / (height_m^2)
5   ) %>%
6   select(BMI, everything()) %>%
7   slice_max(order_by = BMI, n = 3)

```

6.4.2. Operar sobre varias columnas a la vez: across()

Cuando la misma operación de resumen debe repetirse sobre varias columnas (por ejemplo, la media de todas las variables numéricas), `across()` evita escribir una línea por columna:

```

1 # Selección explícita de columnas
2 starwars %>%
3   group_by(species) %>%
4   summarise(across(c(height, mass, birth_year), ~ mean(.x, na.rm = TRUE)))
5
6 # Selección por tipo, usando los "selectores" de select()
7 starwars %>%
8   group_by(species) %>%
9   summarise(across(where(is.numeric), ~ mean(.x, na.rm = TRUE)))

```

Podemos combinar `group_by()`, `filter()` (para excluir grupos con muy pocas observaciones), `across()`, `arrange()` y `tidyr::drop_na()` en un único pipeline legible:

```

1 library(tidyr)
2
3 Wide_Starwars <-
4   starwars %>%
5     group_by(homeworld) %>%
6     filter(n() > 1) %>%
7     summarise(across(where(is.numeric), ~ mean(.x, na.rm = TRUE))) %>%
8     arrange(desc(birth_year)) %>%
9     drop_na()
10
11 Wide_Starwars

```

6.5. Reestructurar tablas: tidyr

Un mismo conjunto de datos puede representarse en formato **ancho** (una columna por variable, como `Wide_Starwars` arriba) o **largo** (una columna que identifica la variable y otra con su valor). Ninguno de los dos formatos es “el correcto” de forma absoluta: el formato ancho suele ser más legible para personas, mientras que el formato largo es el que exigen la mayoría de las funciones de `ggplot2` (Capítulo 7) y de modelado estadístico.

6.5.1. `pivot_longer()`: de ancho a largo

```
1 Long_Starwars <-
2   Wide_Starwars %>%
3   pivot_longer(cols = c(height:birth_year), names_to = "Variable", values_to = "Valores")
4
5 Long_Starwars
```

6.5.2. `pivot_wider()`: de largo a ancho

`pivot_wider()` deshace exactamente la transformación anterior:

```
1 Long_Starwars %>%
2   pivot_wider(names_from = "Variable", values_from = "Valores")
```

6.6. Verbos de dos tablas: combinar conjuntos de datos

Con frecuencia, la información relevante para un análisis está repartida en más de una tabla (por ejemplo, mediciones ambientales por estación y riqueza de especies por estación, en un estudio ecológico). Los verbos `*_join()` de `dplyr` combinan dos tablas a partir de una o varias columnas clave compartidas.

```
1 library(readr)
2
3 Mean_Galapagos <- read_csv("INPUT/DATA/Galapagos_summary.csv")
4 Species <- read_csv("INPUT/DATA/Species_Richness_PerSite.csv")
```

Antes de combinar dos tablas, conviene inspeccionar si las claves de unión coinciden exactamente (mismos niveles, misma capitalización, mismos espacios):

```
1 levels(factor(Mean_Galapagos$Island))
2 levels(factor(Species$Island))
```

6.6.1. left_join(): conservar todas las filas de la tabla izquierda

```
1 left_join(Mean_Galapagos, Species, by = c("Island", "Station", "distance"))
```

6.6.2. full_join(): conservar todas las filas de ambas tablas

`full_join()` conserva todas las filas de ambas tablas, rellendo con NA donde no exista una coincidencia. Es la opción adecuada cuando ninguna de las dos tablas es “la principal”, como al combinar datos de accidentalidad con datos de anchura de carreteras, indexados por el nombre de la vía:

```
1 Acc_Car <- read_delim("INPUT/DATA/accidentalidad-por-carreteras.csv",
2                       delim = ";", escape_double = FALSE, trim_ws = TRUE)
3 Ancho_Car <- read_delim("INPUT/DATA/anchura-de-carreteras.csv",
4                          delim = ";", escape_double = FALSE, trim_ws = TRUE)
5
6 Accidentes_total <-
7   Acc_Car %>%
8   select(`AÑO`, T.RED, NOMBRE, LONG.:IG) %>%
9   full_join(
10     Ancho_Car %>% select(PR:CARRETERA, LONGITUD:ESTAC.\nAFORO`),
11     by = c("NOMBRE" = "CARRETERA")
12   )
```

Nótese el uso de `by = c("NOMBRE" = "CARRETERA")`: cuando la columna clave tiene **distinto nombre** en cada tabla, se lo indicamos explícitamente en lugar de dejar que `dplyr` intente adivinar una unión “natural” por nombres coincidentes (algo que, si no se revisa, puede combinar tablas por columnas equivocadas sin ningún aviso).

Con la tabla combinada, podemos explorar, por ejemplo, la relación entre la anchura de la calzada y el índice de mortalidad (IM) por accidentes de tráfico:

```
1 library(ggplot2)
2
3 Accidentes_total %>%
4   filter(IM > 0) %>%
5   ggplot(aes(x = `ANCHURA\nASF\n(m.)`, y = IM)) +
6   geom_point(na.rm = TRUE) +
7   geom_smooth(na.rm = TRUE) +
8   labs(x = "Anchura de la carretera (m)", y = "Índice de mortalidad (IM)",
9         subtitle = "Relación entre anchura de la vía y accidentalidad mortal") +
10  theme_classic()
```


6.7. Guardar los resultados

Como ya vimos en el Capítulo 5, cualquier tabla resultante de un pipeline de `dplyr/tidyr` se guarda exactamente igual que cualquier otro `data.frame/tibble`:

```
1 Mayores_BMI <-  
2   starwars %>%  
3   mutate(height_m = height / 100, BMI = mass / (height_m^2)) %>%  
4   select(BMI, everything()) %>%  
5   slice_max(order_by = BMI, n = 3)  
6  
7 write_csv(x = Mayores_BMI, file = "OUTPUT/DATA/Mayores_BMI.csv")
```

7 Visualización de la información

7.1. Objetivos

Un análisis correcto que no se comunica con claridad pierde buena parte de su valor. Este capítulo presenta `ggplot2` (Wickham, 2016; Wickham, Chang, et al., 2026), el paquete de visualización del `tidyverse`, construido sobre la **gramática de gráficos** de Leland Wilkinson: un sistema declarativo en el que un gráfico se especifica describiendo *qué* relación entre variables queremos mostrar, dejando que `ggplot2` resuelva los detalles de renderizado.

```
1 library(tidyverse)    # incluye ggplot2
2 data("mpg")           # eficiencia de combustible de distintos modelos de coche
```

7.2. La gramática de gráficos

Todo gráfico de `ggplot2` se construye combinando un pequeño número de componentes independientes, que se suman con el operador `+`:

- **Datos** (`data`): siempre un `data.frame`/`tibble`, nunca vectores sueltos.
- **Maapeo estético** (`aes()`): qué variable de los datos controla qué propiedad visual (posición `x/y`, color, forma, tamaño...).
- **Geometría** (`geom_*()`): la forma visual que representa cada observación (puntos, barras, líneas...).
- **Transformación estadística** (`stat_*()`): un resumen calculado a partir de los datos antes de dibujarlos (un conteo, en el caso de `geom_bar()`; un ajuste de suavizado, en `geom_smooth()`).
- **Escala** (`scale_*()`): cómo se traduce un valor de los datos a una propiedad visual concreta (qué color exacto corresponde a cada categoría).
- **Sistema de coordenadas** (`coord_*()`): cartesiano por defecto, pero también polar, con ejes invertidos, etc.
- **Facetas** (`facet_*()`): dividir el gráfico en subgráficos según los niveles de una variable categórica.
- **Tema** (`theme()`): todo el aspecto visual no vinculado directamente a los datos (tipografía, fondo, cuadrícula).

Definir únicamente los datos y el maapeo estético, sin geometría, produce un lienzo vacío con los ejes ya escalados, pero sin ninguna marca visual todavía:

```
1 ggplot(data = mpg, aes(x = displ, y = hwy))
```

7.3. Gráfico de dispersión

Añadiendo `geom_point()` obtenemos un gráfico de dispersión, la geometría más directa para mostrar la relación entre dos variables continuas:

```
1 ggplot(data = mpg, aes(x = displ, y = hwy)) +
2   geom_point()
```

7.3.1. Mapeo estético del color

El color de los puntos puede fijarse a un valor **constante** (fuera de `aes()`) o **mapearse** a una variable de los datos (dentro de `aes()`); esta distinción es una de las que con más frecuencia confunde a quien empieza con `ggplot2`:

```
1 # Color fijo: no depende de los datos
2 ggplot(mpg, aes(x = displ, y = hwy)) +
3   geom_point(colour = "deeppink")
4
5 # Color mapeado a una variable categórica
6 ggplot(mpg, aes(x = displ, y = hwy)) +
7   geom_point(aes(colour = factor(cyl)))
```

7.3.2. Añadir un ajuste: `geom_smooth()`

Superponer una línea de tendencia ayuda a interpretar la relación subyacente entre las variables, y `ggplot2` permite comparar directamente distintos métodos de ajuste ("`loess`" para un suavizado local no paramétrico, "`lm`" para un modelo lineal):

```
1 ggplot(mpg, aes(x = displ, y = hwy)) +
2   geom_point() +
3   geom_smooth(method = "loess", colour = "blue") +
4   geom_smooth(method = "lm", formula = y ~ poly(x, 1), colour = "red")
```

Podemos incluso ajustar un modelo **independiente por cada nivel** de una variable categórica, simplemente mapeando esa variable también dentro de `geom_smooth()`:

```
1 ggplot(mpg, aes(x = displ, y = hwy)) +
2   geom_point(aes(colour = factor(cyl))) +
3   geom_smooth(method = "lm", aes(colour = factor(cyl)))
```

7.4. Gráfico de barras

```
1 data("diamonds")
2
3 ggplot(diamonds, aes(x = cut)) +
4   geom_bar()
```

7.4.1. Proporciones y relleno

`geom_bar()` con `position = "fill"` normaliza cada barra a una altura total de 1, útil para comparar la composición interna entre categorías del eje X sin que el tamaño absoluto de cada grupo distraiga la comparación; `position = "dodge"` coloca las subcategorías una junto a otra en lugar de apiladas:

```
1 ggplot(diamonds, aes(x = cut)) +
2   geom_bar(aes(fill = clarity), colour = "black")           # apilado (por defecto)
3
4 ggplot(diamonds, aes(x = cut)) +
5   geom_bar(aes(fill = clarity), position = "fill")          # proporciones (comparación)
6
7 ggplot(diamonds, aes(x = cut)) +
8   geom_bar(aes(fill = clarity), position = "dodge")         # lado a lado (comparación)
```

7.4.2. Coordenadas polares

Cambiar el sistema de coordenadas, de cartesiano a polar, transforma un gráfico de barras en un gráfico tipo *coxcomb* (variante circular del clásico de Florence Nightingale para mortalidad hospitalaria):

```
1 ggplot(diamonds) +
2   geom_bar(aes(x = cut, fill = cut), show.legend = FALSE, width = 1) +
3   theme(aspect.ratio = 1) +
4   labs(x = NULL, y = NULL) +
5   coord_polar()
```

7.5. Facetas: pequeños múltiples

Las facetas dividen un único gráfico en una rejilla de subgráficos, uno por cada nivel (o combinación de niveles) de una o más variables categóricas, evitando saturar un único panel con demasiadas dimensiones a la vez:

```

1 ggplot(mpg, aes(x = displ, y = hwy)) +
2   geom_point(aes(colour = factor(cyl))) +
3   facet_wrap(vars(drv), nrow = 1)
4
5 # Dos variables de faceta a la vez, con "margins" que añade el total de cada fila/columna
6 ggplot(mpg, aes(x = displ, y = hwy)) +
7   geom_point(aes(colour = factor(cyl))) +
8   facet_grid(class ~ drv, scales = "free", margins = TRUE)

```

7.6. Etiquetas

`labs()` centraliza el texto de ejes, título, subtítulo y leyendas en una única llamada:

```

1 ggplot(mpg, aes(x = displ, y = hwy)) +
2   geom_point(aes(colour = class, shape = factor(cyl))) +
3   labs(
4     x = "Cilindrada (litros)",
5     y = "Rendimiento en autopista (millas/galón)",
6     title = "Rendimiento en autopista",
7     colour = "Tipo de \nvehículo",
8     shape = "Cilindros \n(número)"
9   )

```

7.6.1. Etiquetas de texto dentro del gráfico

Cuando queremos identificar observaciones individuales directamente sobre el gráfico (en lugar de mediante una leyenda de color), `geom_text()` superpone una etiqueta de texto por punto. Con muchas observaciones, las etiquetas se solapan; `check_overlap = TRUE` elimina las que colisionarían, y el paquete `ggrepel` (Slowikowski, 2026) va más allá, desplazando automáticamente las etiquetas para evitar solapamientos sin perder ninguna:

```

1 library(ggrepel)
2
3 ggplot(mpg, aes(displ, hwy)) +
4   geom_point(size = 0.4, colour = "red") +
5   geom_text_repel(aes(label = manufacturer))

```

7.7. Límites de los ejes: recortar frente a hacer zoom

Un error frecuente es confundir dos formas, aparentemente equivalentes, de “acercar” un gráfico a una región de interés:

```

1 # lims()/xlim()/ylim(): ELIMINA los datos fuera del rango antes de calcular
2 # cualquier estadístico (como el ajuste de geom_smooth())
3 ggplot(mpg, aes(hwy, displ)) +
4   geom_point() +
5   stat_smooth() +
6   lims(x = c(19, 24))
7
8 # coord_cartesian(): conserva TODOS los datos para el cálculo, y solo
9 # recorta la VENTANA de visualización final
10 ggplot(mpg, aes(hwy, displ)) +
11   geom_point() +
12   stat_smooth() +
13   coord_cartesian(xlim = c(19, 24), expand = TRUE)

```

! Por qué esta diferencia importa

`lims()` elimina físicamente las observaciones fuera del rango indicado **antes** de que `stat_smooth()` calcule el ajuste, de modo que la línea de tendencia se recalcula únicamente con los datos visibles, pudiendo cambiar de forma sustancial (incluso de signo de la pendiente). `coord_cartesian()` calcula el ajuste con **todos** los datos y solo decide, al final, qué porción de ese resultado se muestra. Para “hacer zoom” sin alterar ningún cálculo estadístico subyacente, `coord_cartesian()` es casi siempre la opción correcta.

7.8. Temas

El tema controla el aspecto del gráfico que no depende de los datos: fondo, cuadrícula, tipografía. `ggplot2` incluye varios temas predefinidos:

```

1 p <- ggplot(mpg, aes(hwy, displ)) + geom_point() + stat_smooth()
2
3 p + theme_classic() # ejes sin cuadrícula, fondo blanco
4 p + theme_minimal() # cuadrícula suave, sin marco
5 p + theme_void()    # elimina ejes y cuadrícula por completo

```

7.9. Guardar gráficos

Un gráfico de `ggplot2` no queda “fijado” hasta que se asigna a un objeto; imprimirlo (ejecutar su nombre) simplemente lo dibuja en el dispositivo gráfico activo. `ggsave()` guarda el último gráfico dibujado —o uno indicado explícitamente con `plot`— controlando tamaño y resolución de salida:

```

1 Final_plot <-
2   ggplot(mpg, aes(x = displ, y = hwy)) +
3   geom_point(aes(colour = class, shape = factor(cyl))) +
4   labs(x = "Cilindrada (litros)", y = "Rendimiento (millas/galón)",
5         title = "Rendimiento en autopista")
6
7   ggsave(
8     filename = "Car_yield_highway.jpeg",
9     plot = Final_plot,
10    path = "OUTPUT/Figures",
11    scale = 0.5, width = 40, height = 20, units = "cm", dpi = 320
12  )

```

Fijar explícitamente `width`, `height`, `units` y `dpi` (en lugar de dejar el tamaño por defecto del dispositivo activo) es imprescindible para que una misma figura tenga un aspecto consistente al incluirla, por ejemplo, en el documento del seminario evaluable (Capítulo 2) o en un artículo científico.

8 Consultas RDF y SPARQL

8.1. Objetivos

Los capítulos anteriores trabajaron siempre con datos **tabulares**: filas de observaciones y columnas de variables. La Web Semántica propone un modelo de datos distinto, pensado para enlazar información entre organizaciones sin necesidad de acordar de antemano un esquema tabular común: **RDF** (*Resource Description Framework*), que representa el conocimiento como un grafo de **tripletas** sujeto-predicado-objeto (Cyganiak et al., 2014). Este capítulo muestra cómo construir y consultar grafos RDF desde R con `rdflib` (Boettiger, 2024), y cómo interrogar *endpoints* SPARQL públicos como DBpedia y Wikidata.

8.2. Del modelo tabular al modelo de tripletas

8.2.1. De un `data.frame` a tripletas

Cualquier tabla puede reinterpretarse como un conjunto de tripletas: cada celda de la tabla es una afirmación de la forma “la fila *X* tiene, para la variable *Y*, el valor *Z*”. `tidyr::gather()` (o su sucesor `pivot_longer()`, Capítulo 6) hace esta transformación explícita:

```
1 library(rdflib)
2 library(dplyr)
3 library(tidyr)
4 library(tibble)
5
6 data(mtcars)
7
8 car_triples <-
9   mtcars %>%
10    rownames_to_column("Model") %>%
11    gather(attribute, measurement, -Model)
12
13 head(car_triples)
```

El resultado tiene tres columnas que corresponden exactamente a los tres elementos de una tripleta RDF:

- `Model` → **sujeto** (*subject*): la entidad de la que se habla.

- `attribute` → **predicado** (*predicate*): la propiedad medida.
- `measurement` → **objeto** (*object*): el valor de esa propiedad.

Aunque parezca una tabla igual que cualquier otra, **no es un conjunto de datos *tidy*** en el sentido de Capítulo 6: mezcla en una sola columna (`attribute`) variables de naturaleza completamente distinta (cilindrada, peso, potencia...), precisamente porque su propósito ya no es el análisis estadístico directo, sino la representación como grafo.

8.2.2. Identificadores URI

El nombre `Model` de la tabla anterior identifica cada fila mediante el nombre del modelo de coche, que no es único a nivel global (dos conjuntos de datos distintos podrían usar el mismo nombre para entidades distintas). Para publicar datos en la Web de forma que puedan enlazarse sin ambigüedad con datos de otras fuentes, cada sujeto —y, habitualmente, cada predicado— se identifica con una **URI** (*Uniform Resource Identifier*) global:

```
1 iris_triples <-
2   iris %>%
3   rowid_to_column("subject") %>%
4   mutate(subject = paste0("http://example.com/iris#", subject)) %>%
5   gather(key = predicate, value = object, -subject) %>%
6   mutate(predicate = paste0("http://example.com/iris#", predicate)) %>%
7   mutate(object = as.numeric(object))
```

i URI frente a URL

Toda URL es una URI, pero no toda URI es una URL. Una **URL** (*Uniform Resource Locator*) localiza un recurso indicando cómo acceder a él (un protocolo, un servidor, una ruta). Una **URI** simplemente lo *identifica*, sin garantizar que sea descargable: un DOI, un ISBN o el propio `http://example.com/iris#1` de nuestro ejemplo son URI que, en este caso, no resuelven a ninguna página real. En Linked Data se prefiere que las URI sean también URL “resolubles” (“*dereferenciable*”), de forma que visitarlas con un navegador o un cliente HTTP devuelva información adicional sobre el recurso, pero la especificación RDF no lo exige (Cyganiak et al., 2014).

8.2.3. Tipos de dato (*datatype URIs*)

Al forzar los valores a tripletas de texto, perdemos su tipo original: un 5.1 numérico puede acabar representado como la cadena "5.1". RDF resuelve esta pérdida asociando explícitamente cada literal con una **URI de tipo de dato**, tomada del vocabulario de esquemas XML (*XML Schema*, XSD): "5.1"~`http://www.w3.org/2001/XMLSchema#double` es, sin ambigüedad, un número de precisión doble, y no una cadena de texto que se parece a un número.

8.3. El paquete `rdflib`

Hasta ahora, nuestras “tripletas” eran simplemente `data.frame` con tres columnas bien elegidas. `rdflib` (Boettiger, 2024) añade un objeto de clase `rdf`, respaldado por la biblioteca C `redland`, que sí entiende semánticamente el grafo (y permite consultarlo con SPARQL, más adelante).

```

1  rdf1 <- rdf()
2
3  base <- "http://example.com/iris#"
4
5  rdf1 %>%
6    rdf_add(subject = paste0(base, "obs1"),
7            predicate = paste0(base, "Sepal.Length"),
8            object = 5.1)

```

8.3.1. Prefijos y CURIE

Escribir la URI completa en cada tripleta es tedioso y poco legible. Un **CURIE** (*Compact URI Expression*) abrevia una URI larga mediante un prefijo: si `iris:` se define como equivalente a `http://example.com/iris#`, entonces `iris:Sepal.Length` es una forma compacta de referirse a la misma propiedad, exactamente igual que un espacio de nombres (*namespace*) en XML.

8.3.2. Serialización: varios formatos, un mismo grafo

Un mismo grafo RDF puede escribirse en varias sintaxis intercambiables (**serializaciones**). `rdflib` permite cambiar el formato de impresión con `options(rdf_print_format = ...)`:

```

1  doc <- system.file("extdata/example.rdf", package = "redland")
2  rdf1 <- rdf_parse(doc, format = "rdfxml")
3
4  options(rdf_print_format = "rdfxml") # XML con espacios de nombres
5  rdf1
6
7  options(rdf_print_format = "turtle") # sintaxis compacta, pensada para lectura humana
8  rdf1
9
10 options(rdf_print_format = "jsonld") # RDF representado como JSON
11 rdf1

```

8.3.2.1. JSON-LD: RDF sobre JSON

La última de estas serializaciones, **JSON-LD** (Sporny et al., 2020), cierra el círculo con el Capítulo 5: es JSON “normal y corriente” —analizable con `jsonlite::fromJSON()` sin ninguna

herramienta especial—, al que se añaden dos claves reservadas que le dan semántica de grafo: `@id` (la URI del sujeto) y `@context` (qué vocabulario usar para interpretar cada clave del objeto, y permitir así que dos documentos JSON-LD que usan nombres de clave distintos se refieran, sin ambigüedad, al mismo predicado global). El paquete `jsonld` (Ooms, 2024) implementa las operaciones estándar sobre este formato:

```
1 library(jsonld)
2
3 # Añadir un @context a un grafo ya serializado en JSON-LD
4 rdf_serialize(rdf1, "example.json", "jsonld") %>%
5   jsonld_compact(context = '{"@vocab": "http://purl.org/dc/elements/1.1/"}')
```

No todo conjunto de datos RDF se presta a una representación tabular directa. Un ejemplo con una estructura más rica:

```
1 ex <- system.file("extdata/person.json", package = "rdflib")
2 cat(readLines(ex), sep = "\n")
3
4 rdf2 <- rdf_parse(ex, "jsonld")
5
6 # jsonld_flatten() "desanida" el grafo: cada nodo (incluidos los nodos en
7 # blanco, como la dirección postal) queda al mismo nivel, identificado por su
8 # propio @id
9 jsonld_flatten(ex, context = "http://schema.org")
10
11 # jsonld_frame() hace el camino inverso: reconstruye una vista anidada a
12 # partir de un "marco" (frame) que especifica qué tipo de nodo debe quedar en
13 # la raíz del documento resultante
14 jsonld_flatten(ex) %>%
15   jsonld_frame('{"@type": "http://schema.org/Person"}') %>%
16   jsonld_compact(context = "http://schema.org")
```

Las representaciones anidada y aplanada de un mismo grafo JSON-LD son equivalentes en contenido; la diferencia es puramente de conveniencia según qué operación queramos hacer a continuación (leer un registro completo, o consultar el grafo entero).

8.3.3. De tablas a grafos, y de vuelta

`rdflib` incluye un atajo, `as_rdf()`, para convertir directamente un `data.frame` en un grafo RDF sin pasar por el paso manual de `gather()` + `rdf_add()`:

```
1 x1 <- as_rdf(iris, NULL, "iris:")
2 x2 <- as_rdf(cars, NULL, "mtcars:")
3
4 rdf3 <- c(x1, x2) # combina ambos grafos en uno solo
```

8.4. Consultas SPARQL

SPARQL es el lenguaje de consulta estándar para grafos RDF (Harris & Seaborne, 2013), con una sintaxis deliberadamente cercana a SQL, pero que en lugar de filas de una tabla recorre patrones de tripletas sobre el grafo.

8.4.1. Consultas sobre un grafo en memoria

Cuando el grafo ya reside en memoria (como `rdf3`, construido arriba), `rdflib::rdf_query()` ejecuta la consulta directamente, sin salir de R:

```

1  sparql <- '
2  SELECT ?Species ?Sepal_Length ?Sepal_Width ?Petal_Length ?Petal_Width
3  WHERE {
4    ?s <iris:Species>      ?Species .
5    ?s <iris:Sepal.Width> ?Sepal_Width .
6    ?s <iris:Sepal.Length> ?Sepal_Length .
7    ?s <iris:Petal.Length> ?Petal_Length .
8    ?s <iris:Petal.Width>  ?Petal_Width
9  }'
10
11 iris2 <- rdf_query(rdf3, sparql)
```

8.4.2. Consultas a *endpoints* SPARQL remotos

Buena parte del interés de la Web Semántica está en consultar grafos que **no** son nuestros: bases de conocimiento públicas como DBpedia (una versión estructurada de Wikipedia) o Wikidata. Estos servicios exponen un ***endpoint* SPARQL**: una URL a la que se envía la consulta por HTTP y que devuelve los resultados, habitualmente en JSON.

Aviso: paquetes retirados de CRAN

Las prácticas originales de este curso consultaban estos *endpoints* con los paquetes **SPARQL** y **WikidataQueryServiceR**. **Ambos han sido retirados de CRAN** (SPARQL en 2022; WikidataQueryServiceR en 2026) y ya no se mantienen activamente, aunque siguen siendo instalables manualmente. A continuación mostramos primero el enfoque original (por fidelidad histórica y porque puede seguir apareciendo en código de terceros) y, después, una alternativa moderna con **httr2** que no depende de ningún paquete huérfano.

8.4.2.1. Enfoque original: los paquetes SPARQL y WikidataQueryServiceR

```

1 # El paquete SPARQL fue retirado de CRAN; puede instalarse desde el Archive:
2 # install.packages(
3 #   "https://cran.r-project.org/src/contrib/Archive/SPARQL/SPARQL_1.16.tar.gz",
4 #   repos = NULL, type = "source"
5 # )
6 library(SPARQL)
7
8 endpoint <- "https://dbpedia.org/sparql"
9
10 query_example <- "SELECT * WHERE {
11   ?athlete rdfs:label 'Alexia Putellas'@en ;
12   dbo:number      ?number ;
13   dbo:birthPlace ?place .
14 }"
15
16 QD <- SPARQL(url = endpoint, query = query_example)
17 DF <- as_tibble(QD$results)

```



```

1 # WikidataQueryServiceR fue retirado de CRAN; puede instalarse desde GitHub:
2 # devtools::install_github("wikimedia/wikidataQueryServiceR")
3 library(WikidataQueryServiceR)
4
5 gatos <- query_wikidata('
6 SELECT ?item ?itemLabel WHERE {
7   ?item wdt:P31 wd:Q146.    # instancia de "gato doméstico"
8   SERVICE wikibase:label { bd:serviceParam wikibase:language "[AUTO_LANGUAGE], en". }
9 }')

```

8.4.2.2. Enfoque recomendado: httr2 + jsonlite

El **protocolo SPARQL 1.1** (Harris & Seaborne, 2013) no exige ningún cliente especializado: es una petición HTTP GET (o POST) ordinaria, con la consulta en el parámetro `query` y la cabecera `Accept: application/sparql-results+json` para pedir la respuesta en JSON. Con `httr2` (Wickham, 2024) para la petición y `jsonlite` (Sección 5.4) para analizar la respuesta, no dependemos de ningún paquete específico de SPARQL:

```

1 library(httr2)
2 library(jsonlite)
3
4 consultar_sparql <- function(endpoint, query) {
5   resp <-

```

```

6     request(endpoint) %>%
7     req_url_query(query = query) %>%
8     req_headers(Accept = "application/sparql-results+json") %>%
9     req_perform()
10
11 resultado <- resp_body_json(resp, simplifyVector = FALSE)
12
13 # La forma JSON de una respuesta SPARQL siempre sigue el mismo esquema:
14 # resultado$head$vars      -> nombres de las variables pedidas en el SELECT
15 # resultado$results$bindings -> una lista por fila, con un {type, value} por variable
16 variables <- unlist(resultado$head$vars)
17
18 filas <- lapply(resultado$results$bindings, function(fila) {
19   valores <- lapply(variables, function(v) {
20     if (!is.null(fila[[v]])) fila[[v]]$value else NA_character_
21   })
22   names(valores) <- variables
23   as_tibble(valores)
24 })
25
26 bind_rows(filas)
27 }

```

Con esta única función, ya reutilizable, repetimos la consulta a DBpedia:

```

1 consultar_sparql(
2   endpoint = "https://dbpedia.org/sparql",
3   query = "SELECT * WHERE {
4     ?athlete rdfs:label 'Alexia Putellas'@en ;
5     dbo:number      ?number ;
6     dbo:birthPlace ?place .
7   }"
8 )

```

Y la consulta a Wikidata, sin depender de WikidataQueryServiceR:

```

1 consultar_sparql(
2   endpoint = "https://query.wikidata.org/sparql",
3   query = 'SELECT ?item ?itemLabel WHERE {
4     ?item wdt:P31 wd:Q146.
5     SERVICE wikibase:label { bd:serviceParam wikibase:language "[AUTO_LANGUAGE], en". }
6   }'
7 )

```

Este patrón —petición HTTP con `httr2`, respuesta en JSON analizada con `jsonlite`— reaparecerá en el Capítulo 9 al consultar la API de PubMed, y es, en general, el camino más robusto para integrar en R **cualquier** API web moderna, tenga o no un paquete de R dedicado y mantenido.

9 Prácticas PubMed

9.1. Objetivos

Cerramos el libro donde probablemente empiece la mayoría de los seminarios del curso: la búsqueda sistemática de literatura biomédica. Este capítulo cubre la consulta programática de PubMed con `pubmedR` ([Aria, 2026](#)), el análisis bibliométrico del corpus resultante con `bibliometrix` ([Aria & Cuccurullo, 2017, 2026](#)), la construcción de nubes de palabras a partir de los resúmenes, y la gestión de la bibliografía citada dentro del propio documento RMarkdown/Quarto del seminario.

9.2. Consultas programáticas a PubMed

9.2.1. Claves de acceso y *E-utilities*

PubMed se consulta programáticamente a través de las *E-utilities* de NCBI ([Sayers, 2022](#)), la misma familia de API REST que sustenta, por debajo, al paquete `pubmedR`. El acceso sin clave está limitado a un número reducido de peticiones por segundo; con una clave de API personal (obtenida en la [página de configuración de la cuenta de NCBI](#)), el límite aumenta sustancialmente:

```
1 library(pubmedR)
2 library(bibliometrix)
3
4 api_key <- "TU_API_KEY" # o api_key <- NULL si no se dispone de una
```

9.2.2. Construir la consulta

La consulta se redacta con la misma sintaxis que en la [búsqueda avanzada](#) del propio sitio web de PubMed, y se copia literalmente a un objeto de R:

```
1 query <- "((pollution[Title]) AND (air[Title])) AND (rural[Title])"
```


9.2.3. Número de resultados antes de descargar

Antes de descargar los metadatos completos, conviene comprobar cuántos artículos devuelve la consulta —contrastando la cifra con la propia web de PubMed— y, si el número es muy elevado, acotar la búsqueda:

```
1 res <- pmQueryTotalCount(query = query, api_key = api_key)
2 res$total_count
3 res$query_translation # cómo ha interpretado NCBI la consulta
```

Limitad el volumen para un ejercicio

Si la consulta devuelve más de 500 artículos, es recomendable acotarla (añadiendo más términos, o un rango de fechas) antes de continuar. Para los ejercicios del curso, un corpus de 250-300 artículos es más que suficiente para ilustrar el análisis bibliométrico sin tiempos de descarga excesivos.

9.2.4. Descarga de metadatos

```
1 D <- pmApiRequest(query = query, limit = res$total_count, api_key = api_key)
```

`pmApiRequest()` devuelve una lista con cinco componentes: `data` (los metadatos en bruto, en formato XML), `query` y `query_translation` (la consulta original y la interpretada por NCBI), `records_downloaded` y `total_counts`.

9.2.5. De XML a tabla

```
1 M <- pmApi2df(D)
2 str(M)
```

A partir de este punto, `M` es un `tibble` ordinario, manipulable con todas las herramientas del Capítulo 6.

9.3. Análisis bibliométrico con *bibliometrix*

bibliometrix (Aria & Cuccurullo, 2017) es una herramienta especializada en cienciometría: en lugar de analizar el contenido de cada artículo individualmente, analiza patrones **agregados** sobre toda la colección bibliográfica (autoría, colaboración, evolución temporal, estructura temática).

9.3.1. Importar el corpus al formato de bibliometrix

```
1 M1 <- convert2df(D, dbsource = "pubmed", format = "api")
```

9.3.2. Comprobar la integridad de los metadatos

No todos los campos bibliográficos están siempre disponibles (algunas revistas no reportan filiación institucional completa, por ejemplo). `missingData()` cuantifica qué proporción de registros tiene cada campo relleno, antes de basar ningún análisis en un campo con demasiados huecos:

```
1 com <- missingData(M1)
2 com$mandatoryTags
```

9.3.3. Métricas bibliométricas principales

```
1 results <- biblioAnalysis(M1, sep = ";")
2 S <- summary(results, k = 10, pause = FALSE)
```

`summary()` resume la producción científica anual, los manuscritos más citados, la autoría y los países más productivos, y las palabras clave más relevantes; `plot()` genera automáticamente las figuras correspondientes.

9.3.4. Estructura conceptual: análisis de co-palabras

El análisis de co-palabras (*co-word analysis*) mapea la estructura temática de la colección a partir de qué palabras clave aparecen juntas con más frecuencia, mediante análisis de correspondencias:

```
1 CS <- conceptualStructure(
2   M1, field = "ID", method = "MCA",
3   minDegree = 10, clust = 5, stemming = FALSE,
4   labelsize = 15, documents = 20, graph = FALSE
5 )
6
7 plot(CS$graph_terms)
8 plot(CS$graph_dendogram)
```

9.4. Nubes de palabras a partir de los resúmenes

Una nube de palabras es una forma rápida —aunque estadísticamente poco rigurosa— de obtener una primera impresión visual de los términos más frecuentes en un conjunto de resúmenes.

9.4.1. Enfoque con PubMedWordcloud

PubMedWordcloud (Fan, 2019) es un paquete específico para este propósito, aunque no se actualiza desde 2019 y utiliza gráficos base de R (no ggplot2):

```
1 library(PubMedWordcloud)
2
3 pmids <- getPMIDs(author = "Yan-Hui Fan", dFrom = 2007, dTo = 2020, n = 10)
4 abstracts <- getAbstracts(pmids)
5 cleanAbs <- cleanAbstracts(abstracts)
6
7 plotWordCloud(cleanAbs, min.freq = 2, scale = c(2, 0.3))
```

9.4.2. Enfoque tidyverse: tidytext + ggplot2

tidytext (Silge & Robinson, 2016) aplica los mismos principios *tidy* de dplyr/tidyr a la minería de texto: cada palabra individual pasa a ser una **fila** de un **tibble** (una palabra = una observación), lo que permite reutilizar directamente count(), filter(), anti_join() y ggplot2 en lugar de aprender una API específica de análisis de texto. Partiendo de los mismos abstracts descargados con pubmedR (M\$AB, en el objeto M de la sección anterior):

```
1 library(tidytext)
2 library(ggwordcloud)
3 library(stringr)
4
5 palabras_frecuentes <-
6   tibble(abstract = M$AB) %>%
7   unnest_tokens(output = word, input = abstract) %>%
8   anti_join(stop_words, by = "word") %>% # elimina palabras vacías ("the", "and"... )
9   filter(!str_detect(word, "[0-9]+$")) %>% # elimina tokens puramente numéricos
10  count(word, sort = TRUE) %>%
11  slice_max(n, n = 100)
12
13 ggplot(palabras_frecuentes, aes(label = word, size = n, colour = n)) +
14   geom_text_wordcloud() +
15   scale_size_area(max_size = 14) +
16   scale_colour_gradient(low = "steelblue", high = "firebrick") +
17   theme_minimal()
```

Además de integrarse mejor con el resto del *pipeline* tidyverse del curso, esta segunda vía deja palabras_frecuentes disponible como un tibble ordinario, reutilizable para cualquier otro análisis (por ejemplo, comparar el vocabulario de dos consultas de PubMed distintas con dplyr::anti_join()), algo que el objeto interno de PubMedWordcloud no facilita.

9.5. Gestión de bibliografía en RMarkdown/Quarto

El seminario evaluable del curso (Capítulo 2) exige una sección de referencias en formato APA. RMarkdown y Quarto gestionan esta bibliografía de forma totalmente integrada, sin necesidad de escribir la lista de referencias a mano ni de mantenerla sincronizada manualmente con las citas del texto.

9.5.1. Formato BibTeX y estilos CSL

Las referencias se guardan en un archivo `.bib` (formato BibTeX: un registro por entrada, con una clave única, como `@article{aria2017bibliometrix, ...}`), y el estilo de citación (APA, Vancouver, etc.) se define en un archivo `.csl` (*Citation Style Language*), descargable desde el [repositorio oficial de estilos Zotero](#). Ambos se declaran en la cabecera YAML del documento:

```
1 ---
2 bibliography: referencias.bib
3 csl: apa.csl
4 ---
```

Este mismo libro sigue exactamente este mecanismo: su bibliografía completa está en `referencias.bib`, con el estilo APA aplicado mediante `apa.csl`.

9.5.2. Citar en el texto

Dentro del texto, `[@clave]` inserta una cita con el formato del CSL activo, y genera automáticamente la entrada correspondiente al final del documento:

```
1 El análisis bibliométrico agrega patrones de toda una colección de
2 literatura científica [@aria2017bibliometrix].
3
4 jsonlite ofrece una correspondencia consistente entre JSON y los tipos
5 de datos de R [@ooms2014jsonlite].
6
7 Pueden citarse varias fuentes a la vez separándolas con `;`
8 [@rpkg-jsonlite; @rpkg-tidyjson].
```

que se renderiza como: el análisis bibliométrico agrega patrones de toda una colección de literatura científica (Aria & Cuccurullo, 2017); `jsonlite` ofrece una correspondencia consistente entre JSON y los tipos de datos de R (Ooms, 2014); y pueden citarse varias fuentes a la vez separándolas con ; (Ooms, 2025; Stanley, 2026).

9.5.3. Gestores de bibliografía en la nube

Mantener un archivo `.bib` a mano, copiando referencias una a una, es tedioso y propenso a errores de formato. Gestores como [Zotero](#), [CiteDrive](#) o [BibSonomy](#) permiten construir la bibliografía de forma colaborativa (desde el DOI del artículo, desde PubMed, o manualmente) y exportarla directamente en formato BibTeX. Con CiteDrive, por ejemplo, el archivo `.bib` del proyecto se sirve incluso desde una URL estable:

```
1 download.file(
2   "https://api.citedrive.com/bibtex/project/<id-del-proyecto>/bibliography.bib",
3   destfile = "referencias.bib"
4 )
```

de forma que basta con volver a ejecutar esa descarga —o, en Quarto, simplemente renderizar de nuevo el documento— para que cualquier referencia añadida al gestor en la nube aparezca ya disponible para citar, sin copiar y pegar manualmente entre sistemas.

9.5.4. Generar la lista de referencias

Por último, el documento necesita un punto en el que Quarto/RMarkdown inserte la lista de referencias generada automáticamente a partir de las citas usadas en el texto. En Quarto, basta con un `div` vacío al final del documento:

```
1 # Referencias
2
3 ::: {#refs}
4 :::
```

Referencias

- Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., & Dervieux, C. (2022). *Quarto: An Open-Source Scientific and Technical Publishing System*. Posit Software, PBC. <https://quarto.org/>
- Aria, M. (2026). *pubmedR: Gathering Metadata about Publications, Grants, Clinical Trials from 'PubMed'*. <https://cran.r-project.org/package=pubmedR>
- Aria, M., & Cuccurullo, C. (2017). bibliometrix: An R-tool for comprehensive science mapping analysis. *Journal of Informetrics*, 11(4), 959-975. <https://doi.org/10.1016/j.joi.2017.08.007>
- Aria, M., & Cuccurullo, C. (2026). *bibliometrix: Comprehensive Science Mapping Analysis*. <https://cran.r-project.org/package=bibliometrix>
- Boettiger, C. (2024). *rdflib: Tools to Manipulate and Query Semantic Data*. <https://cran.r-project.org/package=rdflib>
- Bray, T. (2017). *The JavaScript Object Notation (JSON) Data Interchange Format* (RFC 8259). Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC8259>
- Bryan, J. (2017). *Happy Git and GitHub for the useR*. <https://happygitwithr.com/>
- Chacon, S., & Straub, B. (2014). *Pro Git*. Apress. <https://git-scm.com/book/en/v2>
- Couture-Beil, A. (2024). *rjson: JSON for R*. <https://cran.r-project.org/package=rjson>
- CRAN Team. (2026). *XML: Tools for Parsing and Generating XML Within R and S-Plus*. <https://cran.r-project.org/package=XML>
- Cygniak, R., Wood, D., & Lanthaler, M. (2014). *RDF 1.1 Concepts and Abstract Syntax*. World Wide Web Consortium (W3C). <https://www.w3.org/TR/rdf11-concepts/>
- Ecma International. (2017). *ECMA-404: The JSON Data Interchange Syntax*. <https://ecma-international.org/publications-and-standards/standards/ecma-404/>
- Fan, Y. (2019). *PubMedWordcloud: 'Pubmed' Word Clouds*. <https://cran.r-project.org/package=PubMedWordcloud>
- Harris, S., & Seaborne, A. (2013). *SPARQL 1.1 Query Language*. World Wide Web Consortium (W3C). <https://www.w3.org/TR/sparql11-query/>
- Müller, K., & Wickham, H. (2026). *tibble: Simple Data Frames*. <https://cran.r-project.org/package=tibble>
- Ooms, J. (2014). *The jsonlite Package: A Practical and Consistent Mapping Between JSON Data and R Objects*. <https://arxiv.org/abs/1403.2805>
- Ooms, J. (2024). *jsonld: JSON-LD Processor*. <https://cran.r-project.org/package=jsonld>
- Ooms, J. (2025). *jsonlite: A Simple and Robust JSON Parser and Generator for R*. <https://cran.r-project.org/package=jsonlite>
- Sayers, E. (2022). *A General Introduction to the E-utilities*. National Center for Biotechnology Information (NCBI). <https://www.ncbi.nlm.nih.gov/books/NBK25497/>
- Silge, J., & Robinson, D. (2016). tidytext: Text Mining and Analysis Using Tidy Data Principles in R. *Journal of Open Source Software*, 1(3), 37. <https://doi.org/10.21105/joss.00037>

- Slowikowski, K. (2026). *ggrepel: Automatically Position Non-Overlapping Text Labels with 'ggplot2'*. <https://cran.r-project.org/package=ggrepel>
- Sporny, M., Longley, D., Kellogg, G., Lanthaler, M., Champin, P.-A., & Lindström, N. (2020). *JSON-LD 1.1: A JSON-based Serialization for Linked Data*. World Wide Web Consortium (W3C). <https://www.w3.org/TR/json-ld11/>
- Stanley, J. (2026). *tidyjson: Tidy Complex 'JSON'*. <https://cran.r-project.org/package=tidyjson>
- Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis* (2.^a ed.). Springer-Verlag. <https://ggplot2-book.org/>
- Wickham, H. (2019). *Advanced R* (2.^a ed.). CRC Press / Chapman & Hall. <https://adv-r.hadley.nz/>
- Wickham, H. (2020). *The tidyverse style guide*. <https://style.tidyverse.org/>
- Wickham, H. (2024). *httr2: Perform HTTP Requests and Process the Responses*. <https://cran.r-project.org/package=httr2>
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- Wickham, H., & Bryan, J. (2026). *readxl: Read Excel Files*. <https://cran.r-project.org/package=readxl>
- Wickham, H., Çetinkaya-Rundel, M., & Golemund, G. (2023). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data* (2.^a ed.). O'Reilly Media. <https://r4ds.hadley.nz/>
- Wickham, H., Chang, W., Henry, L., Pedersen, T. L., Takahashi, K., Wilke, C., Woo, K., Yutani, H., & Dunnington, D. (2026). *ggplot2: Create Elegant Data Visualisations Using the Grammar of Graphics*. <https://cran.r-project.org/package=ggplot2>
- Wickham, H., François, R., Henry, L., Müller, K., & Vaughan, D. (2026). *dplyr: A Grammar of Data Manipulation*. <https://cran.r-project.org/package=dplyr>
- Wickham, H., Hester, J., & Bryan, J. (2026). *readr: Read Rectangular Text Data*. <https://cran.r-project.org/package=readr>
- Wickham, H., Hester, J., & Ooms, J. (2026). *xml2: Parse XML*. <https://cran.r-project.org/package=xml2>
- Wickham, H., Vaughan, D., & Girlich, M. (2025). *tidyr: Tidy Messy Data*. <https://cran.r-project.org/package=tidyr>
- Xie, Y., Allaire, J. J., & Golemund, G. (2018). *R Markdown: The Definitive Guide*. CRC Press / Chapman & Hall. <https://bookdown.org/yihui/rmarkdown/>